Cuadernillo Sparkies

Noviembre 2025

Contents

1 Data Structures **2**
- 1.1 Disjoint Set Union Find (DSU) . . . 2
- 1.2 Segment Tree . . . 2
  - 1.2.1 Lazy Recursive . . . 2
  - 1.2.2 Iterative . . . 2
- 1.3 Lichao . . . 3
- 1.4 Fenwick Tree . . . 3
- 1.5 Treap . . . 3
- 1.6 Mo Algo . . . 4
- 1.7 tipsNtricks . . . 5

2 Trees **5**
- 2.1 Lowest Common Ancestor (LCA) . . . 5
- 2.2 Tree tricks . . . 5

3 Graph **5**
- 3.1 random shit . . . 5
  - 3.1.1 shortest paths . . . 5
  - 3.1.2 bridges . . . 6
  - 3.1.3 floyd-warshal . . . 6
- 3.2 Dinitz . . . 6
- 3.3 Blossoms . . . 6
- 3.4 Hungarian . . . 7
- 3.5 Euler Circuit . . . 8
- 3.6 2 SAT . . . 8
- 3.7 clique . . . 8

4 Strings **8**
- 4.1 Z function . . . 8
- 4.2 Prefix function . . . 9
  - 4.2.1 Applications . . . 9
- 4.3 Suffix Array . . . 10
- 4.4 Aho-Corasick automata . . . 10
- 4.5 Manacher . . . 10
- 4.6 Hash . . . 11
- 4.7 Booth algorithm for minimalShift . . . 11

5 Algebra and friends **11**
- 5.1 FPS . . . 12
- 5.2 Berlekamp-Massey . . . 12
- 5.3 Bostan-Mori . . . 12
- 5.4 Tonelli-Shanks . . . 13
  - 5.4.1 Fast Walsh Hadamard Transform (FWH) . . . 13
  - 5.4.2 SOS-DP . . . 13
  - 5.4.3 divisibility convolution . . . 13
- 5.5 extended gcd . . . 13
- 5.6 Xor Basis . . . 14
- 5.7 prime stuff . . . 14
- 5.8 Floor Sums . . . 15
- 5.9 Max-plus convolution . . . 15
- 5.10 sieve . . . 16

6 Combinatorics **16**
- 6.1 Properties . . . 16
  - 6.1.1 n Choose k . . . 16
  - 6.1.2 Stirling God . . . 16
  - 6.1.3 Formal Power Series . . . 16
  - 6.1.4 Modular Arithmetic . . . 17
  - 6.1.5 Chinese Remainder Theorem . . . 17
  - 6.1.6 Lagrange Interpolation . . . 17
  - 6.1.7 Catalan numbers . . . 17
- 6.2 Techniques . . . 17

7 Geometry **17**
- 7.1 geomTemplate . . . 17
- 7.2 Convex Hull (2D) . . . 18

8 linAlg **18**
- 8.1 LP Simplex . . . 18
- 8.2 matrix primitives . . . 19
- 8.3 slae . . . 19

9 Cosas random **20**
- 9.1 Calcular $\lfloor log_2 x \rfloor$ . . . 20
- 9.2 My tipsNtricks stuff . . . 20
- 9.3 template . . . 20
  - 9.3.1 c++ . . . 20
  - 9.3.2 Python . . . 21
- 9.4 Exhaustive testing . . . 21

1 Data Structures

1.1 Disjoint Set Union Find (DSU)

```
/* Autor: Oscar Vargas Pabon
DSU que permite hacer rollback de la ultima operacion (como un stack)
Funciona en tiempo O(lg n) porque no se puede hacer 'path-compression'
Util en cosas como offline dynamic connectivity
Probado con impl de offlineDynamicConnectivity en
        https://codeforces.com/gym/100551/problem/A
Me gusto un problema de offlineDynamicConnectivity en
        https://www.luogu.com.cn/problem/P5625
Remember that to do XOR path queries (given weighted edges) I do
        int path_sum(int x){int x=0;if(pi[x]!=x)x=wg[x]^path_sum(pi[x]);return
             x;}
        And in merge I modify
                wg[x]=edge_weight^path_sum(ex)^path_sum(ey); pi[x]=y; cnt[y]+=
                     cnt[x];
        where (ex,ey) is the added edge
Can I extend it to + path queries or * path queries????
*/ struct DSU{ vector<int> pi,cnt,stc;
        DSU(int n=0){ pi.resize(n);cnt.resize(n);stc.resize(0);
                for(int i=0;i<n;++i)pi[i]=i,cnt[i]=1;
        } int get_pi(int nd){return pi[nd]==nd?nd:get_pi(pi[nd]);}
        bool merge(int x,int y){
                x=get_pi(x);y=get_pi(y);
                if(x!=y){ if(cnt[x]>cnt[y])swap(x,y);
                        pi[x]=y; cnt[y]+=cnt[x];
                        stc.push_back(x);
                }else stc.push_back(-1);
                return x!=y;
        } void rollback(int am=1){
                while(am--){ assert(!stc.empty());
                        int x=stc.back(); stc.pob();
                        if(x!=-1){ cnt[pi[x]]-=cnt[x];
                                pi[x]=x; }
                }
        } };
```

1.2 Segment Tree

1.2.1 Lazy Recursive

```
/*
Autor: Oscar Vargas Pabon
Material de referencia para ICPC

tested in https://codeforces.com/problemset/problem/2117/H

l -> left tree bound, r -> right tree bound
ql -> left op bound, qr -> right op bound

For historic sums -> queries:
        1 -> A[i]+=x for l<=i<=r
        2 -> B[i]+=A[i] for 0<=i<n
        3 -> \sum_{i=l}^r B[i]
Do stuff with timestamps. Both lazy and data have 'tot,acum,time'
        normalizing to the actual time is tot'= acum*(ac_time-time)+tot
        Lazy keeps the value of a single element in acum while Data keeps the
             whole range
        so update is tot'=acum*(ac_time-time)+tot +(o.acum*(ac_time-o.time)+o.
             tot)*(r-l+1)
Idea applied in subtask of https://www.luogu.com.cn/problem/P11659
*/
struct Lazy{
        int vl;
        Lazy(int v=0):vl(v){};

        inline void combine( const Lazy &o ){
                // unir con otro lazy (o estaba en el padre)
                vl+=l.vl;
        } inline pair<Lazy,Lazy> separe(int l,int r){
                // separar en izquierda y derecha
                return {*this,*this};
        }
}; struct Data {
        int sum;
        Data(int s=0) : sum(s) {};
        inline Data operator+(const Data&o)const{
                return Data(sum+o.sum);
        }inline void update(const Lazy&o,int l,int r){
                // modificar el dato
                sum+=(r-l+1)*o.vl;
        }
}; struct SegT{ vector<Data> tree; vector<Lazy> tag; int t_size;
        SegT()=default; SegT( const vector<Data>&arr ){
                t_size=arr.size();
                tag.resize(t_size*2); tree.resize(t_size*2);
                --t_size;
                raw_build( arr );
        } inline static void calc_cons(int ind,int l,int r,int&m,int&rind){
                m = (l+r)/2; rind = ind+2*(m-l+1);
        }
void raw_build(const vector<Data>&arr,int ind=0,int l=0,int r=-1){
        if(r==-1)r=t_size;
    /* funcion recursiva auxiliar que construye el segTree */
        if ( l == r ) tree[ind] = arr[l]; // caso base
        else { int m,rind; calc_cons(ind,l,r,m,rind);
                raw_build( arr, ind+1, l, m ); // hijo izquierdo
                raw_build( arr, rind, m+1, r ); // hijo derecho
                tree[ind] = tree[ind+1] + tree[rind];
        } tag[ind] = Lazy(); // siempre en neutro
} inline void push( int ind, int l, int r ) {
/* Empuja el valor del tag a los nodos mas profundos */
        // if(tag[ind]==Lazy())return;//may be usefull to skip work
        tree[ind].update(tag[ind],l,r); if(l<r){
                int m, rind; calc_cons(ind,l,r,m,rind);
                pair<Lazy,Lazy> sd=tag[ind].separe(l,r);
                tag[ind+1].combine(sd.first);
                tag[rind].combine(sd.second);
        } tag[ind] = Lazy(); // para no sobrecontar
} void update(int ql,int qr,Lazy vl,int ind=0,int l=0,int r=-1){
        if(r==-1)r=t_size;
/* actualizo los vlores en arr[lx],...,arr[rx] por arr[lx]+=vl,...,arr[rx]+=vl
        en el arbol en tiempo O(lg n) */
        bool done=0;
        if(ql<=l&&r<=qr)tag[ind].combine(vl),done=1;
        push( ind, l, r );
        if(!(r<ql||qr<l)&&!done){
                int m,rind; calc_cons(ind,l,r,m,rind);

                update(ql,qr,vl,ind+1,l   ,m);
                update(ql,qr,vl,rind  ,m+1,r);
                tree[ind]=tree[ind+1]+tree[rind]; }
} Data query( int ql, int qr, int ind=0, int l=0, int r=-1 ) {
        if(r==-1)r=t_size;
/* Hago una query de arr[lx]+...+arr[rx] en tiempo O(lg n) */
        Data res; push( ind, l, r );
        if ( ql <= l && r <= qr ) res = tree[ind];
        else if ( !(r<ql||qr<l) ) {
                int m,rind; calc_cons(ind,l,r,m,rind);
                res=query(    ql     ,min(qr,m),ind+1,l   ,m)+
                    query(max(ql,m+1),    qr    ,rind  ,m+1,r);
        } return res; }
};
```

1.2.2 Iterative

```
struct Data{
    int mn,mx;
```

```
    Data(int n=inf,int x=-inf):mn(n),mx(x){};
    Data operator +(const Data&o)const{
        return Data( min<int>(mn,o.mn),max<int>(mx,o.mx) ); }
// remember I can modify it to receive range-update and point-query
}; struct SegT{// in a fairly straightforward manner
        vector<Data> tree;int n;
        SegT()=default;
        SegT(int nn):n(nn){ tree.resize(2*nn); }
        SegT(const vector<Data>&arr){
                n=arr.size();tree.resize(2*n);//O(n)
                for(int i=0;i<n;++i)tree[i+n]=arr[i];
                for(int i=n-1;i;--i)tree[i]=tree[i*2]+tree[i*2+1];
        } void update( int x,const Data&val ) {
        /* Modifica el valor en arr[x] segun la representacion del arbol en O(
            lg n) */
                tree[x+=n]=val;
                for(x/=2;x;x/=2)tree[x]=tree[x*2]+tree[x*2+1];
        } Data query( int l, int r )const{
        /* Responde a la query arr[l] + ... + arr[r] en tiempo O(lg n) */
                Data res;
                for (l+=n,r+=n;l<=r;l/=2,r/=2) {
                        if( l&1 )res=res+tree[l++];
                        if(!(r&1))res=res+tree[r--];
                } return res; } };
```

1.3 Lichao

```
// Author: Oscar Vargas Pabon. Motivated by Judival
// tested in https://codeforces.com/problemset/problem/932/F
// Min convex hull trick implementation
// requires Point from ..\geometry\geotemplate.cpp supporting dot product (*)
// and '=='. P(m,b) = <P.x,P.y> = L(x)=m*x+b
// It works by assigning an id to each tree (-1 is code for no tree).
// It supports update, alias add line;
// querys and merge trees. The tree has size O(n) for n:linear functions added
// and query/update time O(lgn), merge time O(nlg m) for m being the tree
    being merged to

// DONT take out the 'tmp'. Turns out there is a weird bug in GCC in which
// the <variable>=<expression> evaluates first <variable> and if evaluating
// <expression> changes the position in memory of <variable> then it breaks
template<typename Tpt,lint tl=lint(-1e9),lint tr=lint(1e9),Tpt inf=Tpt(1e18)>
struct LiChao_dyn{
        static Tpt eval(const Point<Tpt>&ln,Tpt x){return Point<Tpt>(x,1)*ln;}
        struct Node{
                Point<Tpt> ln; array<int,2> chld;
                void reset(){ln={0,inf};chld={-1,-1};}
                Node(Point<Tpt> aln={0,inf}){reset();ln=aln;}
        };
        vector<Node> pool;int pool_sz; inline void reset()noexcept{pool_sz=0;}
        constexpr LiChao_dyn(int preres=0){reset();pool.reserve(preres);}
        inline int new_tree(){if(int(pool.size())==pool_sz)pool.push_back(Node
            ());
                pool[pool_sz].reset(); int rs=pool_sz++; return rs;}

        int update(int nd, Point<Tpt>ln, lint l=tl,lint r=tr){
                if(nd==-1) nd=new_tree();
                const lint m=(l+r)/2ll; Point<Tpt>&aln=pool[nd].ln;
                if(eval(ln,m)<eval(aln,m))swap(ln,aln);
                if(ln==Point<Tpt>(0,inf)||l>=r)return nd;
                if      (eval(ln,l)<eval(aln,l)){
                        const int tmp=update(pool[nd].chld[0],ln, l ,m);
                        pool[nd].chld[0]=tmp;
                } else if(eval(ln,r)<eval(aln,r)){
                        const int tmp=update(pool[nd].chld[1],ln,m+1,r);
                        pool[nd].chld[1]=tmp;
                } return nd;
        } Tpt query(int nd,Tpt x, lint l=tl,lint r=tr){
                if(nd==-1)return inf;
                const lint m=(l+r)/2ll; Tpt res=eval(pool[nd].ln,x);
                if(x<=m) res=min<Tpt>(res,query(pool[nd].chld[0],x, l ,m));
                else     res=min<Tpt>(res,query(pool[nd].chld[1],x,m+1,r));
                return res;
        } int merge(int to,int from,lint l=tl,lint r=tr){
                if(to==-1)return from;
                else if(from==-1)return to;
                to=update(to,pool[from].ln,l,r); const lint m=(l+r)/2ll;
                const array<lint,2> bndl={l,m+1},bndr={m,r}; for(int i=0;i
                    <2;++i){
                        const int tmp=merge(pool[to].chld[i],pool[from].chld[i
                            ],bndl[i],bndr[i]);
                        pool[to].chld[i]=tmp;
                } return to;
        }
}; LiChao_dyn<lint,lint(-1e5-33),lint(1e5+33)> cht(1e5);
```

1.4 Fenwick Tree

```
/*
Autor: Oscar Vargas Pabon
Material de referencia para ICPC
Lo probe en testing\rangeQtest.cpp
*/
template<typename tint> struct fenw{
        vector<tint>bit; int n;
        fenw(int nn=0){ n=nn;bit.resize(n+1,0);}
        fenw(const vector<tint>&arr) { assert(arr[0]==0);//note arr[0] is
            undefined
                n=arr.size();bit.resize(n+1,0);fill(bit.begin(),bit.end(),tint
                    (0));
                int nxt; for(int i=0;i<=n;++i) {
                        bit[i]+=arr[i];nxt=i+(i&(-i));
                        if(nxt<=n)bit[nxt]+=bit[i]; }
        } void update( int x, tint val ) {
                for(;x<=n;x+=x&(-x))bit[x]+=val;
        } tint query(int l,int r=-1)const{ // [l,r]
                tint res=0; if(r==-1)r=l,l=1; --l;//(0,r]
                for(;l<r;r-=r&(-r))res+=bit[r];
                for(;r<l;l-=l&(-1))res-=bit[l];
                return res;
        }int binlift( tint vl )const{
                int x=0,nxt;//min{i|sum_{j<=i} bit_j > vl}
                for(int exp=ilog2(n+1);exp>=0;--exp){
                        nxt=x|(1<<exp);if(nxt<n&&bit[nxt]<vl)
                                vl-=bit[x=nxt];
                } return x+1;
        } tint raw_query(int x)const{ tint res=0;for(;x;x-=x&(-x))res+=bit[x];
            return res;}
};
```

1.5 Treap

```
/*
Autor: Oscar Vargas Pabon

Notar que este treap construye un max-heap en el atributo 'key'.
No tiene anadido ningun tipo de 'lazy propagation'
Las operaciones asumen indexacion en 0

Notar que en esta version, el valor del nodo (vl) funciona como
        el tamano del subarbol que representa (trabajo sobre una lista
            implicita)

Its a ( max-heap _ ascendent bst )

Testeado en https://www.spoj.com/problems/GSS6/en/ ; https://codeforces.com/
    problemset/problem/1558/D
*/
// std::mt19937_64 rng_64( std::chrono::steady_clock::now().time_since_epoch()
    .count() );
```

```
// asumo el anterior elemento de mi template
namespace treap{
struct Lazy{
    int x;
    Lazy(){x=0;}
    Lazy(int xx):x(xx){};
    void combine(const Lazy&o){
        x+=o.x; // o is a 'parent'
    }
}; struct Data{
        int pref,suf,sum,mx;
        Data(){ pref=suf=mx=-1e9; sum = 0; }
        Data( int vl ) { pref=suf=sum=mx= vl; }
        Data operator +( const Data &o ) const {
                Data res; // NO necesariamente es asociativo
                res.sum = sum+o.sum;
                res.pref=std::max(pref,sum+o.pref);
                res.suf=std::max(o.suf,o.sum+suf);
                res.mx=std::max({mx,o.mx,suf+o.pref});
                return res;
        } void combine(const Lazy&o){
        mx+=o.x;
    }
}; struct Node{
        Data mv; // mv->MyValue;
        Data sv; // sv->SubtreeValue;
        Lazy lt; // lt->LazyTag
        int key, vl; //(key,vl) max-heap by key ; bst by vl
        Node *l,*r; // hijos
        Node( const Data &dt=Data() ){
                key = rng_64()%(1<<31); vl = 1;
                mv=dt;
                sv=dt;
                l=r=NULL;
        }
}; std::ostream &operator<<(std::ostream &os,const Node*rt) {
    auto aux=[&](auto rec, const Node*nd)->void{
        if(!nd)return;
        rec(rec,nd->l); os << nd->mv.sum << '-'; rec(rec,nd->r);
    }; aux(aux,rt); return os;
} inline int gvl(Node*act){return act?act->vl:0;}
inline Data gsv(Node*act){return act?act->sv:Data();}
inline void update(Node*act){if(!act)return;
        act->sv = gsv(act->l) + act->mv + gsv(act->r);
        act->vl = gvl(act->l) +     1   + gvl(act->r);
} inline void push(Node*act){if(!act)return;
    if(act->l)act->l->lz.combine(act->lz);
    if(act->r)act->r->lz.combine(act->lz);
    act->mv.combine(act->lz);
    act->sv.combine(act->lz);
    act->lz=Lazy();
} void join( Node *&t, Node *l, Node *r){
        push(l);push(r);
        if ( !l || !r ) t=(l)?l:r;
        else if ( l->key < r->key ) join(r->l,l,r->l),t=r;
        else join(l->r,l->r,r),t=l;
        update(t);
} void split( Node*t, int x, Node *&l, Node *&r ) {
        push(t);// l=t[0..x); r=t[x..n)
        int lvl=(t)?((t->l)?t->l->vl+1:1):0;
        if (!t) l=r=NULL;
        else if ( lvl <= x ) split(t->r,x-lvl,t->r,r),l=t;
        else split(t->l,x,l,t->l),r=t;
        update(t);
} void insert(Node *&t, int pos, Data vl){
        // a[0..pos]+vl+a(pos..n)
        Node *l,*r;
        split(t,pos,l,r);

        Node *neo= new Node(vl);
        join(t,l,neo);
        join(t,t,r);
} void update(Node *&t, int pos, Data vl){
        Node *l,*m,*r; split(t,pos,l,m); split(m,1,m,r);
        m->mv=vl;
        join(t,l,m); join(t,t,r);
} void remove(Node *&t,int pos){
        // a[0..pos)+a(pos..n)
        Node *l,*m,*r;
        split(t,pos,l,m);
        split(m,1,m,r);
        if(m) delete m;
        join(t,l,r);
} Data query(Node *&t,int ql,int qr){
        // a[ql..qr)
        Node *l,*m,*r;
        split(t,qr,m,r);
        split(m,ql,l,m);

        Data res=m->sv;
        join(t,m,r); join(t,l,t);
        return res;
} Node * build(const std::vector<Data> &arr){
        std::vector<Node*> ms;//MonotonicStack
        for(const Data&act:arr){
                Node *nd=new Node(act),*prv=NULL;
                while(!ms.empty()&&ms.back()->key<=nd->key){
                        prv=ms.back(); ms.pop_back();
                        update(prv);
                } if(!ms.empty())ms.back()->r=nd;
                nd->l=prv;
                ms.push_back(nd);
                update(nd);
        } Node *prv=NULL;
        while(!ms.empty()){
                ms.back()->r=prv;
                prv=ms.back(); ms.pop_back();
                update(prv);
        } return prv;
}
}//end namespace treap
```

1.6 Mo Algo

```
/*
Autor: Oscar Vargas Pabon

Recordar que existe la idea de balanceo (usar sqrt-decomp o otra de modo que
        la update se haga en O(1), aunque la query pueda empeorar, pues el
            algoritmo hace
        mas updates que queries)


Asumo que T(REM) y T(ADD) son los tiempos de remover y anadir un indice
        del global que maneja MO.

Idea 1: usar bloque de tamano sqrt(n). Genera tiempo O((n+q)sqrt(n))
Idea 2: usar bloque de tamano n/sqrt(q). Genera tiempo O(nsqrt(q))
        source: https://codeforces.com/blog/entry/61203?#comment-451304
Idea 3: Usar orden de hilbert. Genera tiempo O(nsqrt(q))
        source: https://codeforces.com/blog/entry/61203
        impl:https://codeforces.com/blog/entry/61203?#comment-1064868

Nota: La idea 1 (o tal vez 2 tambien) puede no funcionar muy bien cuando se
    requiere hacer un balanceo
Probado con https://www.spoj.com/problems/DQUERY/
Otro problema interesante https://codeforces.com/problemset/problem/2006/D
```

```
If its not easy to implement REM, I can always use the buckets idea and then
    increase monotonically the
        right border. For the left border, we always keep a 'snapshot' before
            starting to move to the left
        like
        1 -> while(r<ac.r)add(++r);
        2 -> snapshot()
        3 -> l=l_buck;while(l>ac.l)add(--l)
        4 -> restore()
        Issue is that it can be really slow and some cases where both r-l<=
            sqrt{n} are edge-cases
*/
uint64_t hilbertorder(uint64_t x, uint64_t y) {
        // https://codeforces.com/blog/entry/61203?#comment-1064868
    const uint64_t logn = __lg(max(x, y) * 2 + 1) | 1;
    const uint64_t maxn = (1ull << logn) - 1;
    uint64_t res = 0; for (uint64_t s = 1ull << (logn - 1); s; s >>= 1) {
        bool rx = x & s, ry = y & s;
        res = (res << 2) | (rx ? ry ? 2 : 1 : ry ? 3 : 0);
        if (!rx) { if (ry) x ^= maxn, y ^= maxn;
            swap(x, y); }
    } return res;
} struct Query{
    int ind, l, r, ord;
    Query()=default;
    Query(int i,int ll,int rr): ind(i),l(ll),r(rr){};
    bool operator < ( const Query &o ) const{ return ord<o.ord; }
}; const int method=0;
vector<int> mo_algo( int n, vector<Query> &query ) {
        const int q = query.size();
        // idea 1 -> O(n\sqrt{q})
        if constexpr(method==0)for(Query &qu:query)qu.ord=hilbertorder(qu.l,qu
            .r);
        else{ int block_size;
          if constexpr(method==1){ // idea 2 -> O(n\sqrt{q})
            int sqrt=1; while(sqrt*sqrt<q)++sqrt;
            block_size=n/sqrt;
          } else { // idea 3 ->O((n+q)\sqrt{n})
            block_size=1; while ( block_size*block_size < n ) ++block_size;
          } block_size=max(block_size,1); for(Query &qu:query){
        int bl_ind=qu.l/block_size;
        qu.ord=(bl_ind&1)?n-qu.r:qu.r;
        qu.ord+=bl_ind*n;
      }
        } sort(query.begin(),query.end());
        int l = 0, r = -1;
        /// Llenar esta parte con lo necesario del problema
        vector<int> res(q);
        vector<int> all(n,0);
        int cnt=0;
        function<void(int)> add = [&]( int x ){
                if ( !all[a[x]] ) ++cnt;
                ++all[a[x]];
        };
        function<void(int)> rem = [&]( int x ) {
                --all[a[x]];
                if ( !all[a[x]] ) --cnt;
        };
        //// termina la parte del llenado
    for ( const Query &act : query ){
        while ( r > act.r ) rem(r--);
        while ( r < act.r ) add(++r);
        while ( l < act.l ) rem(l++);
                while ( l > act.l ) add(--l);

                res[act.ind] = cnt; // esto depende del ejercicio
        } return res;
}
```

1.7 tipsNtricks

Remember doing binary lifting (sparse table) is posible (precalculating all jumps)

Remember doing coordinate compression is possible

2 Trees

2.1 Lowest Common Ancestor (LCA)

```
int pi[max_n][2],dpt[max_n];
void build_lca(const vector<vector<int>>&t,int nd=0,int p=0,int d=0){
        if(!nd)pi[0][0]=pi[0][1]=0;
        dpt[nd]=d;pi[nd][0]=p; // this ensures somehow O(lgn) query time
        int pp=pi[p][1],ppp=pi[pp][1]; if(dpt[p]-dpt[pp]==dpt[pp]-dpt[ppp])
                pi[nd][1]=ppp;
        else pi[nd][1]=p,bl[nd][1]=arr[nd];
        for(int e:t[nd])if(e!=p)build_lca(t,e,nd,d+1);
} int lca(int u,int v){ if(dpt[u]<dpt[v])swap(u,v);
        while(dpt[u]>dpt[v])u=pi[u][dpt[pi[u][1]]>=dpt[v]];
        while(u!=v){ int jmp=pi[u][1]!=pi[v][1];
                u=pi[u][jmp];v=pi[v][jmp];
        } return u;//notice I am already on u
} inline int dist(int u,int v){// counts distance in edges
    const int p=lca(u,v); //+1 to count in nodes
    return dpt[u]+dpt[v]-2*dpt[p]; }
```

2.2 Tree tricks

```
/* Author: Oscar Vargas Pabon
Some random tree-tricks
*/  ////// this trick was first used in
// https://codeforces.com/gym/102201/problem/F
int tin[max_n],tout[max_n]; // taking in-out times (my version)
int get_time(const vector<vector<int>>&t,int nd=0,int p=0,int tm=0){
        tin[nd]=tm++;for(int e:t[nd])if(e!=p)tm=get_time(t,e,nd,tm);
        return tout[nd]=tm;
} // when having in and out times like this, one can query path sums
int a[max_n],acm[max_n]; // in the following way
void built(int n){ // the indexing is a bit fragile, so take that into
        for(int i=0;i<=n;++i)acm[i+1]=0; // account.
        for(int i=0;i<n;++i)acm[tin[i]+1]+=a[i],acm[tout[i]+1]-=a[i];
        for(int i=0;i<n;++i)acm[i+1]+=acm[i];
} inline int root_sum(int nd){ return acm[tin[nd]+1]; }
inline int path_sum(int u,int v){
        return root_sum(u)+root_sum(v)-2*root_sum(lca(u,v))+a[lca(u,v)];
} // in Some cases, it can be better to do this via dfs and mantaining
// some shared memory. The idea is to keep at all times the answers
// from the root to the actual node
```

Remember centroid decomposition exists and generates a tree of height $O(\log n)$. Furthermore, remember HLD also exists and that it allows for queries in $O(\log nT(n))$ where $T(n)$ is the time to query some range data structure.

Remember that for all $A \subseteq V$ we have that $H = \{lca(u,v)|u,v \in A\}$ is computable by ordering via dfs in times and doing $\{lca(A_i, A_{i+1})\}_{i=0}^{|A|-1}$

3 Graph

3.1 random shit

3.1.1 shortest paths

Remember 0-1 BFS exists. Remember $\delta(u,v) \leq \delta(u,k)+\delta(k,v)$ for any k and that there exists k that makes it equality (unless $(u,v) \in E$). Remember that A^* exists and it requires the over-estimation function to fulfill $\delta(u,v) \geq \delta(u,k)+OE(k,v)$ where OE is the over-estimation. Remember that dijkstra (as other methods) can span an arborescence or a dag (of used edges).

3.1.2 bridges

Remember the algorithm of bridges we have studied mantains the time of arrival of each node and the minimum time it looked on its path, in a way its

$$mini(v) = min \begin{cases} time(v) & \\ mini(c) & \text{Where } c \text{ is } v\text{'s child in the DFS tree} \\ time(e) & \text{Where } (v,e) \text{ represents a back-edge} \end{cases}$$

For each child in the DFS tree there is an articulation point in v iff $mini(c) \geq time(v)$; now (v,c) is a bridge iff $mini(c) > time(v)$.

3.1.3 floyd-warshal

We have the dp function $dst(k+1,u,v) = min(dst(k,u,v), dst(k,u,k) + dst(k,k,v))$. Remember it can be optimized to $O(n^2)$ memory by the relaxation step being safe. For only connectivity it can be further optimized by bitset my love.



3.2 Dinitz

Note that it usually works in $O(V^2E)$. In unit networks (they all have capacity of 1 except for source and sink) it works in $O(E\sqrt{v})$. Leaving the implementation as is, it works as dinitz with scaling so complexity on integers is $O(VE \log U)$ where U is the max capacity.

```
/* Author: Oscar Vargas Pabon
Lo probe en https://codeforces.com/problemset/problem/2026/E
Mi implementacion interpreta los valores de 'graph' como indices de 'edges'
Caso general funciona en O(V*E*lgU), donde U es la capacidad maxima
        caso capacidad unitaria funciona en O(min(E*sqrt(E), E*V^(2/3)))
        caso red unitaria (cada nodo se responsabiliza por 1 sola arista de
            capacidad unitaria) funciona en O(E*sqrt(V))
Es facil reducirlo a la version usual de dinitz

Remember, remember. Whenever we have a planar graph, a path of its dual graph
    represents
        bijectively a cut on the original graph. Essentially solves with
            dijkstra min-cut
Anfelesan taught me this one in regards to https://codeforces.com/gym/106178/
    problem/G
*/ template <typename ftype, ftype INF> struct Dinitz{// int ex;
        constexpr static ftype inf(){return INF;}
        struct Edge { int u, v;
                ftype c1, c2, f;// c1 es la capacidad u->v y c2 es para v->u
                Edge()=default;
                Edge( int uu, int vv, ftype cc1, ftype cc2=0 ) : u(uu),v(vv),
                    c1(cc1),c2(cc2),f(0) {};
                inline void push( int nd, ftype pushed ) { // empujar un flujo
                    'pushed' desde 'nd'
                        if ( nd == u ) c1 -= pushed, c2 += pushed, f += pushed
                            ;
                        else c1 += pushed, c2 -= pushed, f -= pushed;
                } inline int to( int nd ) const { // el opuesto a 'nd'
                        return ( u == nd ) ? v : u;
                } inline ftype cap( int nd ) const { // la capacidad desde 'nd
                    '
                        return ( this->u == nd ) ? c1 : c2;
                } inline ftype flow( int nd ) const { // el flujo desde 'nd'
                        return ( u == nd ) ? f : -f; }
        }; // level -> para el 'layered network'
        // blocked-> para omitir vertices 'blocked'
        // ptr -> para omitir aristas 'blocked'
        vector<int> level,ptr,q; vector<bool>blocked;

        vector<vector<int>> graph; vector<Edge> edges;
        int source, sink; Dinitz(int n){
                graph.resize(n); level.resize(n); q.resize(graph.size());
                ptr.resize(n); blocked.resize(n);
        } void add_edge( int u, int v, ftype c1, ftype c2=0 ) {
                // u->v has capacity c1; v->u has capacity c2
                graph[u].push_back( edges.size() );
                graph[v].push_back( edges.size() );
                edges.emplace_back( Edge( u, v, c1, c2 ) );
        } ftype push_dfs( int nd, ftype flow,int ex ) {
// Empuja el flujo por todas las aristas del 'layered graph' que pueda
                if ( nd == sink || flow==0 ) return flow; // ya llegamos o no
                    podemos empujar mas
                ftype push_flow = 0, edge_flow;
                int nxt,&pt=ptr[nd]; while(pt<int(graph[nd].size())&&flow>
                    push_flow ){
                        auto&e=edges[graph[nd][pt]];nxt = e.to(nd);
                        if(!((e.c1+e.c2)>>ex))break;
                        // anadir '(e.cap(nd)>>ex)' para maximo aseguramiento
                        if ( level[nd] < level[nxt] && !blocked[nxt] ) {
                                edge_flow = push_dfs( nxt, min(e.cap(nd),flow-
                                    push_flow),ex );
                                push_flow += edge_flow;
                                e.push( nd, edge_flow );
                        }     pt+=push_flow<flow;
                }blocked[nd]=push_flow<flow; return push_flow;
        } ftype max_flow( ) {
// Hace el maximo flujo del grafo (retorna el maxFlow, las asignaciones quedan
    en las aristas)
//              Funciona en peor caso O(V*E*lgU)
                ftype mx=0;for(const Edge&e:edges){//quitar esto para reducir
                        if(mx<e.c1)mx=e.c1;if(mx<e.c2)mx=e.c2;// a la version
                }int ex=0;while((mx>>ex)>1)++ex;// usual de dinitz
                for(auto&adj:graph)sort(adj.begin(),adj.end(),[&](int a,int b)
                    {
                        return edges[a].c1+edges[a].c2>edges[b].c1+edges[b].c2
                            ;});
                ftype flow=0;for(;ex>=0;--ex)do{
                        fill(all(level),0);level[source]=1;
                        int iq,nq=1;q[iq=0]=source;while(iq<nq&&!level[sink]){
                                const int nd=q[iq++]; for(int eind:graph[nd]){
                                        if(!((edges[eind].c1+edges[eind].c2)>>
                                            ex))break;
                                        const int e=edges[eind].to(nd);
                                        if(!level[e]&&(edges[eind].cap(nd)>>ex
                                            ))
                                                level[e]=level[nd]+1,q[nq++]=e
                                                    ;
                                }
                        } if(!level[sink])continue;
                        fill(all(blocked),0); fill(all(ptr),0);
                        flow+=push_dfs( source, INF, ex );
                } while(level[sink]); return flow; }
};
```

3.3 Blossoms

```
/*
Sacado del comentario de bicsi encontrable en
https://codeforces.com/blog/entry/92339?#comment-810166

Retorna un vector de vertices que tiene -1 si no ha sido considerado en el matching,
    de lo contrario el vertice con el que ha sido 'matcheado'.
Toma tiempo O(V*E).
Probado en <11439 Maximizing the ICPC>.
*/
vector<int> Blossom(vector<list<int>>& graph) {
  int n = graph.size(), timer = -1;
  vector<int> mate(n, -1), label(n), parent(n),
              orig(n), aux(n, -1), q;
  auto lca = [&](int x, int y) {
    for (timer++; ; swap(x, y)) {
      if (x == -1) continue;
      if (aux[x] == timer) return x;
      aux[x] = timer;
      x = (mate[x] == -1 ? -1 : orig[parent[mate[x]]]);
    }
  };
  auto blossom = [&](int v, int w, int a) {
    while (orig[v] != a) {
      parent[v] = w; w = mate[v];
      if (label[w] == 1) label[w] = 0, q.push_back(w);
      orig[v] = orig[w] = a; v = parent[w];
    }
  };
  auto augment = [&](int v) {
    while (v != -1) {
      int pv = parent[v], nv = mate[pv];
      mate[v] = pv; mate[pv] = v; v = nv;
    }
  };
  auto bfs = [&](int root) {
    fill(label.begin(), label.end(), -1);
    iota(orig.begin(), orig.end(), 0);
    q.clear();
    label[root] = 0; q.push_back(root);
    for (int i = 0; i < (int)q.size(); ++i) {
      int v = q[i];
      for (auto x : graph[v]) {
        if (label[x] == -1) {
          label[x] = 1; parent[x] = v;
          if (mate[x] == -1)
            return augment(x), 1;
          label[mate[x]] = 0; q.push_back(mate[x]);
        } else if (label[x] == 0 && orig[v] != orig[x]) {
          int a = lca(orig[v], orig[x]);
          blossom(x, v, a); blossom(v, x, a);
        }
      }
    }
    return 0;
  };
  // Time halves if you start with (any) maximal matching.
  for (int i = 0; i < n; i++)
    if (mate[i] == -1)
      bfs(i);
  return mate;
}
```

3.4 Hungarian

Solves the problem of the max-matching of min-cost given edge-weights on the graph.

```
/*
 * Hungarian Algorithm
 * -------------------------------------------
 *  Entrada:
 *   - Una matriz cuadrada cost_matrix de tamano nXn,
 *     donde cost_matrix[i][j] representa el costo de asignar
 *     el trabajador i al trabajo j.
 *
 *  Salida:
 *   - El costo minimo total de asignacion optima.
 *
 *  Uso:
 *   - Resuelve el "Assignment Problem": asignar n tareas a n agentes
 *     minimizando el costo total (o maximizando utilidad si se invierte el
     signo).
 *   - Complejidad: O(n^3)
 */

#include <bits/stdc++.h>
using namespace std;

class Hungarian {
private:
    vector<vector<int>> cost_matrix; // Matriz de costos
    int n;                           // Tamano de la matriz
    vector<int> u, v, p, way;        // Vectores auxiliares para el algoritmo

public:
    // Constructor: inicializa con la matriz de costos
    Hungarian(const vector<vector<int>> &matrix) : cost_matrix(matrix) {
        n = matrix.size();
        u.resize(n + 1, 0);
        v.resize(n + 1, 0);
        p.resize(n + 1, 0);
        way.resize(n + 1, 0);
    }

    // Funcion principal: devuelve el costo minimo total
    int compute() {
        for (int i = 1; i <= n; ++i) {
            p[0] = i;
            vector<int> minv(n + 1, INT_MAX);
            vector<bool> used(n + 1, false);
            int j0 = 0;
            while (true) {
                used[j0] = true;
                int i0 = p[j0], delta = INT_MAX, j1 = 0;
                for (int j = 1; j <= n; ++j) {
                    if (!used[j]) {
                        int cur = cost_matrix[i0 - 1][j - 1] - u[i0] - v[j];
                        if (cur < minv[j]) {
                            minv[j] = cur;
                            way[j] = j0;
                        }
                        if (minv[j] < delta) {
                            delta = minv[j];
                            j1 = j;
                        }
                    }
                }
                for (int j = 0; j <= n; ++j) {
                    if (used[j]) {
                        u[p[j]] += delta;
                        v[j] -= delta;
                    } else {
                        minv[j] -= delta;
                    }
                }
                j0 = j1;
                if (p[j0] == 0)
                    break;
            }
            while (true) {
                int j1 = way[j0];
```

```
                p[j0] = p[j1];
                j0 = j1;
                if (j0 == 0)
                    break;
            }
        }

        int total_cost = 0;
        for (int j = 1; j <= n; ++j)
            total_cost += cost_matrix[p[j] - 1][j - 1];
        return total_cost;
    }
};
```

3.5 Euler Circuit

Nota: Mi impl puede no funcionar bien para no dirigidos. Upd:nomas use un set para ver cuando no puede visitar algo y ya (para el undirected case)

```
/* Autor: Oscar Vargas Pabon
Esta implementacion destruye el grafo y es solo para dirigidos.
Recordar que solo hay un circuito euleriano si indeg[nd]==outdeg[nd] para
        todos los nodos.
Un camino euleriano se cumple cuando para u,v indeg[u]+1==outdeg[u],
        indeg[v]==outdeg[v]+1 y todos los demas nodos cumplen indeg[nd]==
            outdeg[nd].
        En este caso basta con iniciar la primera solucion de 'euler_circuit'
            en u.
*/ void euc_aux( vector<list<pair<int,int>>> &g, int nd, list<pair<int,int>> &
    res ) {
        if ( g[nd].empty() ) return;
        int neo_nd = g[nd].front().first; val = g[nd].front().second;
        res.pb( pair<int,int>(nd,val) ); g[nd].pop_front();
        euc_aux( g, neo_nd, res );
} list<pair<int,int>> euler_circuit( vector<list<pair<int,int>>> &g ) {
        list<pair<int,int>> res; euc_aux( g, 0, res );
        for ( list<pair<int,int>>::iterator it = res.begin() ; it != res.end()
            ; ++it ){
            if ( g[it->first].empty() ) continue;
            list<pair<int,int>> tmp; euc_aux( g, it->first, tmp );

            for ( const pair<int,int> &act : tmp ) res.insert(it,act);
            while ( !tmp.empty() ) {tmp.pop_front(); --it;}
        } return res; }
```

3.6 2 SAT

For $\wedge_i(x_i \vee y_i)$ generate a graph with nodes $V = \{x_i, \neg x_i, y_i, \neg y_i\}_i$ and edges $(\neg x_i, y_i), (\neg y_i, x_i) \in E$ for all i. Then one should greedily try to take elements from the 'terminal' StronglyConnectedComponents to prevent paths $x_i \rightsquigarrow \neg x_i$ (or viceversa). I still don't understand why the CP-Algorithms implementation work, but here it is (it was slightly adapted).

```
struct TwoSatSolver {// from cp-algorithms
    int n_vars, n_vertices;
    vector<vector<int>> adj, adj_t;
    vector<bool> used, assignment;
    vector<int> order, comp;
    TwoSatSolver(int _n_vars) : n_vars(_n_vars),
    n_vertices(2 * n_vars), adj(n_vertices),
    adj_t(n_vertices), used(n_vertices), order(),
    comp(n_vertices, -1), assignment(n_vars) {
        order.reserve(n_vertices);
    } void dfs1(int v) {
        used[v] = true; for (int u : adj[v]) {
            if (!used[u]) dfs1(u);
        } order.push_back(v);
    } void dfs2(int v, int cl) {
        comp[v] = cl; for (int u : adj_t[v]) {
            if (comp[u] == -1) dfs2(u, cl);
        }
    } bool solve_2SAT() {
        order.clear(); used.assign(n_vertices, false);
        for (int i = 0; i < n_vertices; ++i) {
            if (!used[i]) dfs1(i);
        } comp.assign(n_vertices, -1);
        for (int i = 0, j = 0; i < n_vertices; ++i) {
            int v = order[n_vertices - i - 1];
            if (comp[v] == -1)dfs2(v, j++);
        } assignment.assign(n_vars, false);
        for (int i = 0; i < n_vertices; i += 2) {
            if (comp[i] == comp[i + 1]) return false;
            assignment[i / 2] = comp[i] > comp[i + 1];
        } return true;
    } void add_disjunction(int a, bool na, int b, bool nb) {
        // na and nb signify whether a and b are to be negated
        a = 2 * a ^ na;
        b = 2 * b ^ nb;
        int neg_a = a ^ 1;
        int neg_b = b ^ 1;
        adj[neg_a].push_back(b);
        adj[neg_b].push_back(a);
        adj_t[b].push_back(neg_a);
        adj_t[a].push_back(neg_b);
    } static void example_usage() {
        TwoSatSolver solver(3); // a, b, c
        solver.add_disjunction(0, false, 1, true);  //     a  v  not b
        solver.add_disjunction(0, true, 1, true);   // not a  v  not b
        solver.add_disjunction(1, false, 2, false); //     b  v     c
        solver.add_disjunction(0, false, 0, false); //     a  v     a
        assert(solver.solve_2SAT() == true);
        auto expected = vector<bool>(True, False, True);
        assert(solver.assignment == expected); }
};
```

Notice that this has 'in-built' an implementation for Kosaraju's algorithm for strongly connected components.

3.7 clique

Some optimizations may be nontrivial?

```
int max_clique(const vector<uint64_t> &g){
    int res=0,am=0;// expected to work decently up to |g|=50
    auto backt=[&](auto rec,int i,uint64_t msk)->void{
        int pp=__builtin_popcountll(msk);if(pp+am<=res)return;
        while(msk){ if(msk>>i&1ull){ msk^=1ull<<i; --pp;
                ++am;rec(rec,i+1,msk&g[i]);--am;
                if(pp+am<=res)return;
            }++i;
        } if(res<am)res=am;
    };backt(backt,0,(1ull<<g.size())-1);
    return res;
}// For a diamond free graph, each edge belongs to,
// at most, 1 maximal clique. O(n^2)
vector<vector<int>> chordal_clique(const vector<vector<bool>> &g){
    // I assume adj-matrix representation of g
    int n=g.size();vector<vector<int>> res;
    vector<vector<bool>> vis(n,vector<bool>(n,0));
    rep(i,0,n)rep(j,0,n)if(!vis[i][j] && g[i][j] ){
        vector<int> act={i,j}; //now I search the group
        rep(k,0,n)if(g[i][k]&&g[j][k])act.pb(k);
        for(int u:act)for(int v:act)vis[u][v]=1;
        res.pb(act);
    } return res; }
```

4 Strings

4.1 Z function

Computa $Z : 1..N \to 1..N$ donde $Z(i)$ implica que $str[j] = str[i+j]$ para todo $j \leq Z(i)$ y $str[Z(i)+1] \neq str[i+Z(i)+1]$.

```
/* Autor: Oscar Vargas Pabon
Lo probe en mi carpeta de pruebas
*/ vector<int> zFunction(const string&cad){ // O(n).
//  z[i] -> maximo prefijo comun de cad y cad[i...]
        const int n = cad.size(); vector<int> z(n,0);
        int l=-1,r=-1;for(int i=1;i<n;++i){ z[i]=r-i;
                if(z[i]>z[i-l])z[i]=z[i-l];if(z[i]<0)z[i]=0;
                while(i+z[i]<n&&cad[z[i]]==cad[i+z[i]])++z[i];
                if(i+z[i]>r)r=i+z[i],l=i;
        } return z; }
```

4.2 Prefix function

Computa $\pi : 1..N \to 1..N$ donde $\pi(i)$ implica que $str[1..\pi(i)] = str[i - \pi(i)..i]$ y este es maximal.

```
/* Autor: Oscar Vargas Pabon
Probado en mis biblioteca
*/ vector<int> prefix_function(const string&cad){//O(n)
// pi[i] -> el tamano del mayor prefijo
// de cad que tambien es sujifo de cad[0..i]
        const int n = cad.size(); vector<int> pi( n, 0 );
        for(int i=1;i<n;++i){ pi[i] = pi[i-1];
                while(pi[i]&&cad[i]!=cad[pi[i]])pi[i]=pi[pi[i]-1];
                if(cad[i]==cad[pi[i]])++pi[i];
        } return pi;
} const int let='z'-'a'+1; // este fue probado en
// https://codeforces.com/problemset/problem/1721/E
vector<vector<int>> prefix_automata(const string&cad){//O(n*Σ)
        vector<int> pi=prefix_function(cad);
        vector<vector<int>> fpi(cad.size(),vector<int>(let));
        for(int i=0;i<int(cad.size());++i)for(int j=0;j<let;++j){
        if(i>0&&cad[i]-'a'!=j)
            fpi[i][j]=fpi[pi[i-1]][j];
        else fpi[i][j]=i+(cad[i]-'a'==j);
    }return fpi; }
```

4.2.1 Applications

```
/*
Author: Juan Diego Collazos
Date: Fri Oct  2 19:30:31 2026
*/

/*
The Knuth-Morris-Pratt algorithm
Search for a pattern inside a text.
pattern = string we are searching for
text    = string where we search
O(|pattern| + |text|)
*/
vector<int> kmp(const string& pattern, const string& text) {
    vector<int> ans;
    if (pattern.empty()) return ans;
    vector<int> pi = prefix_function(pattern);
    int j = 0;
    for (int i = 0; i < sz(text); ++i) {
        while (j > 0 && text[i] != pattern[j]) j = pi[j - 1];
        if (text[i] == pattern[j]) ++j;
        if (j == sz(pattern)) {
            ans.push_back(i + 1 - sz(pattern));
            j = pi[j - 1];
        }
    }
    return ans;
}

/*
Counting the number of occurrences of each prefix of s
inside t.
ans[i] = number of occurrences of
         s[0 ... i-1] inside t.
O(|s| + |t|)
*/
vector<int> count_occ_each_prefix(
    const string& s,
    const string& t
) {
    const int n = sz(s);
    vector<int> ans(n + 1);
    if (n == 0) return ans;
    vector<int> pi = prefix_function(s);
    int j = 0;
    for (int i = 0; i < sz(t); ++i) {
        while (j > 0 && t[i] != s[j]) j = pi[j - 1];
        if (t[i] == s[j]) ++j;
        ++ans[j];
        if (j == n) j = pi[j - 1];
    }
    for (int i = n; i > 0; --i) ans[pi[i - 1]] += ans[i];
    return ans;
}

/*
Counting the number of occurrences of each prefix
inside the string itself.
ans[i] = number of occurrences of
         s[0 ... i-1] inside s.
O(n)
*/
vector<int> count_occ_each_prefix_itself(const string& s) {
    const int n = sz(s);
    vector<int> ans(n + 1);
    if (n == 0) return ans;
    vector<int> pi = prefix_function(s);
    /*Each pi[i] tells us that the prefix of length pi[i]
    occurs as a suffix ending at position i.*/
    for (int i = 0; i < n; ++i) ++ans[pi[i]];
    /* Propagate occurrences through the border tree. */
    for (int i = n; i > 0; --i) ans[pi[i - 1]] += ans[i];
    /* Every prefix occurs at least once: as itself.*/
    for (int i = 0; i <= n; ++i) ++ans[i];

    return ans;
}

/*
The number of different substrings in a string.
For every prefix s[0...i]:
    new substrings = length - maximum prefix-function value
                     of reverse(s[0...i])
O(n^2)
*/
lint number_diff_substring(const string& s) {
    const int n = sz(s);
    lint ans = 0;
    string t;
    vector<int> pi;
    for (int i = 0; i < n; ++i) {
        t += s[i];
        string tinv(t.rbegin(), t.rend());
        pi = prefix_function(tinv);
        int mx = *max_element(all(pi));
        ans += sz(t) - mx;
    }
    return ans;
}

/*
```

```
Compressing a string.
Returns the shortest string whose repetition
produces s.
O(n)
*/
string compress_string(const string& s) {
    const int n = sz(s);
    if (n == 0) return "";
    vector<int> pi = prefix_function(s);
    int k = n - pi[n - 1];
    if (n % k == 0) return s.substr(0, k);
    return s;
}
```

4.3 Suffix Array

Computa $A : 1..N \to 1..N$ donde $i < j$ implica $str[A(i)..N] < str[A(j)..N]$. También puede computar $LCP : i..N \to 1..N$ donde $LCP(i)$ identifica el tamaño del prefijo comun más grande entre $str[A(i-1)..N]$ y $str[A(i)..N]$.

```
/* Autor: Oscar Vargas Pabon
Probado en Codeforces -> ITMO -> SuffixArray -> step4 -> A
*/ const char EOS = '#';//end-of-string -> estrictamente menor
//                      a todos los demas del string
inline int m_add(int x,int y,int m){return m-x>y?x+y:y-(m-x);}
inline int m_sub(int x,int y,int m){return x>=y?x-y:x+(m-y);}
vector<int> suffixArray( string &str ) {
/* Halla el suffix-array (SA) en tiempo O(nlg n) */
    str.push_back( EOS ); const int n=str.size(); // ahorra edge-cases
    vector<int> master(n),code(n),newCode(n); {
        // para las 'equivalence classes' cuando solo contienen un caracter
        vector<pair<char,int>> p_master(n);
        for(int i=0;i<n;++i)p_master[i]=pair<char,int>(str[i],i);
        std::sort(p_master.begin(),p_master.end());
        master[p_master[0].second]=0;for(int i=1;i<n;++i){
            if(p_master[i-1].first<p_master[i].first)
                code[p_master[i].second]=code[p_master[i-1].second]+1;
            else code[p_master[i].second]=code[p_master[i-1].second]  ;
        } for(int i=0;i<n;++i)master[i]=p_master[i].second;
    }vector<int> copy,bucket(n);
    int k=1;while(k<n&&code[master.back()]<n-1){ // hace el 'cyclic-shift'
        for(int i=0;i<n;++i)master[i]=m_sub(master[i],k,n); {//  del master
            copy=master;fill(bucket.begin(),bucket.end(),0);// bucket-sort
            for(int i=0;i<n;++i)bucket[code[i]+1]++;//<- equivalent to
            for(int i=1;i<n;++i)bucket[i]+=bucket[i-1];// code[copy[i]+1]
            for(int i=0;i<n;++i)master[bucket[code[copy[i]]]++]=copy[i];
        } newCode[master[0]]=0; for(int i=1;i<n;++i){// creo las nuevas
            newCode[master[i]] = newCode[master[i-1]];//equivalence clases
            if(code[       master[i-1]      ]!=code[       master[i]      ]||
               code[m_add(master[i-1],k,n)]!=code[m_add(master[i],k,n)] )
                ++newCode[master[i]]; // la tupla de codigos es distinta
        } code = newCode; k = ( k << 1 );
    } str.pop_back(); return master; // quito el EOS
} vector<int> lcpArray( vector<int> &sufix, string &str ) {
/* halla el longest-common-prefix array del sufixArray en tiempo O(n)
    Retorna lcp[i] -> lcp de sufix[i] y sufix[i+1] */
    // me ahorra un edge-case
    str.push_back( EOS ); const int n = sufix.size();
    vector<int> lcp(n-1),inv(n); for(int i=0;i<n;++i)inv[sufix[i]]=i;
    int k=0; for(int i=0;i<n-1;++i) {
        if(inv[i]-1<0)continue;// a veces sale de memoria si no pongo
        const int j=sufix[inv[i]-1];// esto xdxdxd
        while(str[i+k]==str[j+k])++k;
        lcp[inv[i]-1] = k;
        --k;if(k<0)k=0;
    } str.pop_back(); return lcp; } // quito EOS
```

4.4 Aho-Corasick automata

```
/*
Autor: Oscar Vargas Pabon
Lo probe en UVA 1449 y https://codeforces.com/gym/106636/problem/A

La implementacion asume reinicializar 'states' - states=vector<Node>(1);
        * Notar que pattern no considera el vertice actual como un posible
            pattern
Remember to use 'next()' and 'pattern()' to do the queries because of the lazy
      stuff.
I assume all patterns are distinct
*/ const int K = 'z'-'a'+1; const char NORM='a';
struct Node { int next[K],output, parent, link, pattern;
        char letter;
        Node ( int p=0, char l=0, int o=-1 ) : parent(p), letter(l),output(o)
            {
                memset( next, -1, sizeof(next) );
                link=pattern=-1; }
}; vector<Node> states(1);
void add_string(const string&str,int strInd=0){//O(|str|)
// anade la cadena 'str' al trie, la marca con 'strInd'
        int act = 0; for(const char letter:str){
                char ch=letter-NORM; if(states[act].next[ch]==-1){
                        states[act].next[ch] = states.size();
                        states.push_back( Node( act, letter ) );
                } act = states[act].next[ch];
        } if(states[act].output<strInd)states[act].output=strInd;
} int link( int act ) ;
int next( int act, char letter ) {
// hallo el siguiente estado a visitar segun el actual y el caracter
        char ch=letter-NORM;if(states[act].next[ch]==-1){
                if(act) states[act].next[ch]=next( link(act), letter );
                else    states[act].next[ch]=0;
        } return    states[act].next[ch];
} int link( int act ) {
// Halla el indice al nodo que representa el 'longest proper suffix' en el
    trie
        if (states[act].link==-1){
                if (act&&states[act].parent)
                        states[act].link=next(link(states[act].parent),states
                            [act].letter);
                else states[act].link=0;
        } return states[act].link;
} int pattern( int act ) {
// Halla el siguiente elemento que pertenece a un patron
        if(states[act].pattern==-1){ const int nxt=link(act);
                if(!nxt)states[act].pattern = 0;
                else if(states[nxt].output!=-1)states[act].pattern=nxt;
                else states[act].pattern = pattern(nxt);
        } return states[act].pattern;
} int main(){
// ejemplo de hallar las ocurrencias de patrones en un texto
        int pt; cin >> pt; // cargo los patrones
        rep(i,0,pt){string cd;cin >> cd; add_string(cd,i);}
        string t; cin >> t;
        vector<int> ocur(pt,0);//cuenta las ocurrencias
        int act=0; rep(i,0,t.size()) {
                act = next(act,t[i]);//salto al nuevo estado
                // esto lo hago en tiempo proporcional a los patrones
                int jmp=(states[act].output==-1)?pattern(act):act;
                while(jmp){++ocur[states[jmp].output];jmp=pattern(jmp);}
        } return 0; }
```

4.5 Manacher

plagiarized epicly from cp-algorithms. manacher_odd returns as $res[i]$ the length of the longest palindromes centered in i (odd palindrome, ranges is $[i - res[i], i + res[i]]$). manacher then modifies the problem to include the even-length palindromes by centering them 'in the middle'. This last function then returns for $i = 2k$ the $res'[i] = 2 \cdot res[k]$ and $i = 2k + 1$ returns $res'[i] = 2 \cdot res[k] + 1$ assuming stuff I dont want to explain.

```
/* Tomado de https://cp-algorithms.com/string/manacher.html
Calcula p[i]=x donde [i-x,i+x] es un palindromo
Notar que no lo calculamos sobre la cadena, sino sobre
    c'='&'+a[0]+'#'+...+a[i]+...+'#'+a[n]+'&'
Si c'[i]='#' me refiero a un palindromo par (de lo contrario un impar) */
vector<int> manacher_odd(string s) {int n = s.size();
    s = "*" + s + "^"; vector<int> p(n + 2);
    int l = 0, r = 1; for(int i = 1; i <= n; i++) {
        if(i <= r) p[i] = min(r - i, p[l + (r - i)]);
        while(s[i - p[i]] == s[i + p[i]]) p[i]++;
        if(i + p[i] > r) l = i - p[i], r = i + p[i];
    } for(int i=0;i+1<p.size();++i)p[i]=p[i+1];
    p.pop_back();p.pop_back(); return p;
}vector<int> manacher(const string&s) {
    string t;for(auto c: s) t += string("|") + c;
    auto res = manacher_odd(t + "|");
    for(int i=0;i+1<res.size();++i)res[i]=res[i+1]-1;
    res.pop_back();if(!res.empty())res.pop_back();
    return res;}
```

4.6 Hash

```
/*
Autor: Oscar vargas Pabon

Nota: en vez de precomputar ebase podriSa usar mpow para hacerlo en O(log n)

NO OLVIDAR INVOCAR 'precompute_prime' ANTES DE USAR

Testeado en; https://codeforces.com/problemset/problem/2132/G
                        https://codeforces.com/problemset/problem/727/E
                        https://codeforces.com/problemset/problem/2209/E
*/

// const int template_limit=1e6;
// mt19937_64 rng_64( chrono::steady_clock::now().time_since_epoch().count() )
    ;
// #define rep(i,strt,end) for(int i = strt ; i !=int(end) ; (int(strt)<int(
    end))?++i:--i )
// template<typename tpow> constexpr tpow mpow(tpow x,lint e,tpow m){tpow res
    =1;while(e){if(e&1ll)res=(res*1ll*x)%m;e>>=1;x=(x*1ll*x)%m;}return res;}
// asumo el anterior de mi template

const int N_HASH=2, PRIME_TESTING=5;
const int LH_B=1e9,RH_B=1<<30;
int base[N_HASH],modul[N_HASH],ebase[template_limit][N_HASH];

void precompute_prime(){
  auto rgen = [&](int lb, int rb ) -> int {
    return lb+(rng_64())%(rb-lb+1);
  }; for ( int i = 0 ; i < N_HASH ; ++i ) base[i]=rgen(2,LH_B);
  int posi; bool prim;
  for ( int i = 0 ; i < N_HASH ; ++i ) {
    do {
      posi = rgen(LH_B,RH_B);
      prim = mpow<int>(2,posi-1,posi)==1;
      for (int j = 0 ; j < N_HASH && prim ; ++j )
        prim=mpow<int>(base[i],posi-1,posi)==1;
      for (int j = 0 ; j < PRIME_TESTING && prim ; ++j )
        prim=mpow<int>(rgen(2,LH_B),posi-1,posi)==1;
    } while ( !prim ) ;
    modul[i] = posi;
  } // precomputar las potencias de la base _ tambien podria hacerlo en O(log
    n) con mpow
  rep(j,0,N_HASH) ebase[0][j]=1;
  rep(i,0,template_limit-1) rep(j,0,N_HASH) ebase[i+1][j]=(ebase[i][j]*1ll*
      base[j])%modul[j];
}

struct Hash{
  int len = 0; array<int,N_HASH> h;
  Hash() :len(0) {rep(i,0,N_HASH)h[i]=0;};
  Hash(const string &cad) { len = cad.size();
    for ( int i = 0 ; i < N_HASH ; ++i ) { h[i]=0;
      for ( char c : cad ) h[i] = ( h[i]*1ll*base[i] + int(c) )%modul[i];
    }
  } Hash( char c ) { len=1;
      rep(i,0,N_HASH)h[i]= int(c)%modul[i];
  } inline bool operator == ( const Hash &o ) const { return len==o.len&&h==o.
      h; }
  inline bool operator !=( const Hash &o ) const { return !(*this==o); }
  inline bool operator < ( const Hash &o ) const { return h<o.h; }
  Hash operator + ( const Hash &o ) const { // *this + o
    Hash res; for ( int i = 0 ; i < N_HASH ; ++i ) {
      res.h[i] = ( ebase[o.len][i]*1ll*h[i] + o.h[i] )%modul[i];
    } res.len = len+o.len; return res;
  } inline void lconc( char c ) { // hago this = c + this
      rep(i,0,N_HASH) h[i] = ( ebase[len][i]*1ll*lint(c) + h[i] ) % modul[i
          ];
      ++len;
  } inline void rconc( char c ) { // hago this=this+c
      rep(i,0,N_HASH) h[i] = ( h[i]*1ll*base[i] + lint(c))%modul[i];
      ++len;
  } inline void ldeconc ( const Hash &l ) { // quito el prefijo l de *this
    for ( int i = 0 ; i < N_HASH ; ++i ) {
      h[i] -=(l.h[i]*1ll*ebase[len-l.len][i] )%modul[i];
        if(h[i]<0)h[i]+=modul[i];
    } len -= l.len;
  } inline void rdeconc( const Hash &r ) { // quito el sufijo r de *this
    for ( int i = 0 ; i < N_HASH ; ++i ) {
      h[i] = ( (h[i]-r.h[i])*1ll*mpow<int>(ebase[r.len][i], modul[i]-2,modul[i
          ]) ) %modul[i];
        if(h[i]<0)h[i]+=modul[i];
    } len -=r.len;
  } Hash operator - ( const Hash&o ) const { Hash res=*this; res.rdeconc(o);
      return res; }
};
```

4.7 Booth algorithm for minimalShift

Returns the index that represents that cad[t:]+ cad[:t] is minimal

```
### returns the shift giving
### lexicographically minimal string
### among transitions. S = S_1 + S_2
### phi : S -> S_2 + S_1
def booth(s: str) -> int:
    s = s + s
    n = len(s) // 2
    i, j, k = 0, 1, 0

    while i < n and j < n and k < n:
        if s[i + k] == s[j + k]:
            k += 1
        elif s[i + k] > s[j + k]:
            i = i + k + 1
            if i <= j:
                i = j + 1
            k = 0
        else:
            j = j + k + 1
            if j <= i:
                j = i + 1
            k = 0
    return min(i, j)
```

5 Algebra and friends

Remember that most of this stuff kind of depends on my modulo_int template, so the modulus often is missing.

5.1 FPS

```
template<typename tint> constexpr int countr_zero_constexpr(tint n){if(n==tint
    (0))return -1;int x=0;while(!(n&(tint(1)<<x)))++x;return x;}
template<typename tint> constexpr tint primitive_root(tint m){
    if ((27ull<<59)+1 == m ) return 5;
    if( (5497558138881ull<<24)+1 == m || 998244353 == m )return 3;
    tint dec[32]={},dind=0,md=m-1; for(tint i=2;i*i<=md;++i)if(md%i==0){
        dec[dind++]=i; while(md%i==0)md/=i;
    } if(md>1)dec[dind++]=md; //tests for g such that
        \forall_{p|(md-1)} g^{(md-1)/p} = 1(mod md)
    bool fnd=0; tint pr=1;while(!fnd){ ++pr; fnd=1;
        for(tint i=0;i<dind&&fnd;++i){ // by properties, this will always end
            if constexpr ( std::is_same_v<tint,unsigned long long> )
                fnd=mpow<tint,__uint128_t>(pr,(m-1)/dec[i],m)!=1;
            else if constexpr ( std::is_same_v<tint,long long> )
                fnd=mpow<tint,__uint128_t>(pr,(m-1)/dec[i],m)!=1;
            else fnd=mpow<tint>(pr,(m-1)/dec[i],m)!=1; }
    } return pr;// std::cerr <<pr << " - primitive root" << endl;
} template<typename tfps> constexpr auto prec_rank_fft_root() {
    constexpr int rank=countr_zero_constexpr(tfps::mod()-1);
    std::array<tfps,rank+1> root; // precompute the roots
    root[rank]=tfps(primitive_root(tfps::mod())).pow((tfps::mod()-1)>>rank);
    for(int i=rank;i>0;--i)root[i-1]=root[i]*root[i];
    return root;
}template<typename tfps> inline constexpr auto rank_fft_root =
    prec_rank_fft_root<tfps>();
template<typename tfps,uint64_t N> inline constexpr tfps fft_root(){
    if constexpr(N<=0ll||countr_zero_constexpr(N)>=int(rank_fft_root<tfps>.
        size()))return tfps(0);
    return rank_fft_root<tfps>[countr_zero_constexpr(N)];
} template<typename tfps,int n>void butterfly(tfps*a){
    constexpr int m=n/2; if constexpr(n<=1)return;
    butterfly<tfps,m>(a); butterfly<tfps,m>(a+m);
    constexpr tfps wlen=fft_root<tfps,n>(); tfps w=1;
    for(int i=0;i<m;++i){tfps u=a[i],v=w*a[i+m];
        a[i]=u+v; a[i+m]=u-v; w*=wlen; }
}template<typename tfps,int n=1<<25>void fft(vector<tfps>&a,bool sd){
    if constexpr(n<=1)return;
    if(n>a.size())fft<tfps,n/2>(a,sd);
    else{ for(int i=1,j=0;i<n;++i){ int bit=n>>1;
            for(;j&bit;bit>>=1)j^=bit;//bit reversal
            j^=bit; if(i<j)swap(a[i],a[j]);
        } butterfly<tfps,n>(a.data()); if(sd){
            tfps ni=tfps(n).inv();//multiplicative
            reverse(a.begin()+1,a.end());// inverse
            for(tfps&ac:a)ac*=ni; } }
} template<typename tfps> vector<tfps>convolution(vector<tfps>A,vector<tfps>B)
    {
    int n=A.size()+B.size()-1,m=1;while(m<n)m*=2;
    A.resize(m,0);B.resize(m,0);
    fft<tfps>(A,0);fft<tfps>(B,0);
    for(int i=0;i<m;++i)A[i]*=B[i];
    fft<tfps>(A,1);A.resize(n);return A;
} template<typename tfps>vector<tfps>inverse(vector<tfps>F,int n){
    assert(F[0]);// G*F=1
    vector<tfps>G={F[0].inv()}; for(int e=2;e<2*n;e<<=1){
        vector<tfps> ac=convolution(G,// gives a ~/2 speedup
                {F.begin(),F.begin()+min<int>(F.size(),e)}
        ); for(tfps&act:ac)act=-act;
        ac[0]+=2;G=convolution(G,ac);
        G.resize(e);
    } G.resize(n);return G; }
```

$$Log(F) = \int \frac{D(F)}{F}$$

EXP

$$G_{<2n>} \equiv G_{<n>}\left(1 - Log(G_{<n>}) + F\right) (\text{mod } x^{2n})$$

POW

$$F^k = Exp(k \cdot Log(F))$$

SQRT

$$G_{<2n>} \equiv 1/2(G_{<n>} + F \cdot G_{<n>}^{-1})(\text{mod } x^{2n})$$

5.2 Berlekamp-Massey

```
// taken from yosupo's submission on https://judge.yosupo.jp/submission/49544
// Returns c such that for the sequence s -> s(n) = sum_{i=1}^{d} c(i)s(n-i)
// where |c|=d ; c(0)=-1 in this impl

template <class T> std::vector<T> berlekamp_massey(const std::vector<T>& s) {
    int n = int(s.size());
    std::vector<T> b = {T(-1)}, c = {T(-1)};
    T y = T(1);
    for (int ed = 1; ed <= n; ed++) {
        int l = int(c.size()), m = int(b.size());
        T x = 0;
        for (int i = 0; i < l; i++) {
            x += c[i] * s[ed - l + i];
        }
        b.push_back(0);
        m++;
        if (x == T(0)) continue;
        T freq = x / y;
        if (l < m) {
            // use b
            auto tmp = c;
            c.insert(begin(c), m - l, T(0));
            for (int i = 0; i < m; i++) {
                c[m - 1 - i] -= freq * b[m - 1 - i];
            }
            b = tmp;
            y = x;
        } else {
            // use c
            for (int i = 0; i < m; i++) {
                c[l - 1 - i] -= freq * b[m - 1 - i];
            }
        }
    }
    return c;
}
```

5.3 Bostan-Mori

To calculate linear recurrences, P/Q then Q is as outputed by Berlekamp-Massey, while $P = B \cdot Q$ for B the base cases of the recurrence.

```
// notice that some speedups like optimizing 1 fft of nQ
// or using recurrences among the even/odd terms of the FFT(F) may give
// greater speedups
template<typename tfps>tfps bostanMori(uint64_t k,vector<tfps>P,vector<tfps>Q)
    {
    // computes [x^k]P/Q in time O(d*lgd*lgk)
    const int d=Q.size(); while(k){ vector<tfps> nQ=Q;
        for(int i=0;i<nQ.size();++i)if(i&1)nQ[i]=-nQ[i];
        P=convolution(P,nQ);Q=convolution(Q,nQ);
        for(int i=0;i<d;++i)Q[i]=Q[i*2];
        Q.resize(d,0); P.resize(d*2+1,0);
        for(int i=0;i<d;++i)P[i]=P[i*2+(k&1ll)];
        P.resize(d); k>>=1;
    } return P[0]/Q[0]; }
```

5.4 Tonelli-Shanks

```
template<typename tfps> int Tonelli_Shanks(tfps a) {
        // retorna c tal que c^2=a(mod tfps::mod())
        //plagiado epicamente de https://judge.yosupo.jp/submission/270105
        const int mod=tfps::mod();
        if (int(a) < 2) return a;
        if (mpow<int>(a, (mod - 1) / 2, mod) != 1) return -1;
        if (mod % 4 == 3) return mpow<int>(a, (mod + 1) / 4, mod);

        tfps b = 3; if (mod != 998244353) {
                while (mpow<int>(b, (mod - 1) / 2, mod) == 1) {
                        b=tfps(int(rng_64()%(mod-3)) + 2);
                }
        } int q = mod - 1,Q = 0;
        while ( !(q&1) ) Q++, q /= 2;

        tfps x = mpow<int>(a, (q + 1) / 2, mod);
        b = mpow<int>(b, q, mod);

        int shift = 2; while ( x*x != a) {
                tfps error= tfps(mpow<int>(a,mod-2,mod))*x*x;
                if (mpow<int>(error, 1 << (Q - shift), mod) != 1) x *= b;
                b *=b; ++shift;
        } return x; }
```

5.4.1 Fast Walsh Hadamard Transform (FWH)

This allows to do $C(k) = \sum_{k=i^j} A_i B_j$ and the bitwise convolutions

```
/* Author: Oscar Vargas Pabon
based loosely on https://atcoder.jp/contests/arc222/submissions/76586635

Tested locally. Remember to do the /n in the end for XOR.
For 'and','or', sd=1 means forward transformation and sd=0 means inverse
    tranformation
*/ template <typename T, int n> void fwt_rc(T*A){
        if constexpr(n==1) return;
        constexpr int m=n/2;
        fwt_rc<T,m>(A);fwt_rc<T,m>(A+m);
        for(int i=0;i<m;++i){
                const T x=A[i],y=A[i+m];
                A[i]=x+y;A[i+m]=x-y;
                // A[i] = (sd)?x+y:x-y;     // and
                // A[i+m] = (sd)?x+y:y-x; // or
        }
}template<typename T,int n=1<<25> void fwt(vector<T>&A){
        if constexpr(n==1)return;//.data() seems powerfull
        if(int(A.size())==n) fwt_rc<T,n>(A.data());
        else fwt<T,n/2>(A);
}template<typename T> void convolution(vector<T> &A, vector<T> B) {
        const int n=A.size(); assert(A.size()==B.size());
        fwt(A); fwt(B);
        for(int i=0;i<n;++i)A[i]*=B[i];
        fwt(A); const T tn=T(n).inv();
        for(int i=0;i<n;++i)A[i]*=tn; }
```

5.4.2 SOS-DP

This allows to do $g(Y) = \sum_{X \subseteq Y} f(X)$ (subset) or $g(Y) = \sum_{Y \subseteq X} f(X)$ (superset) and their inverses happen whenever $tp = 0$. Notice this is another way to calculate the And transformation for FWH and the Or transformation.

```
//SUBSET convolution
for(int e=1;e<n;e*=2)for(int i=0;i<n;++i)
  if(i&e)vec[i  ]+=tp?vec[i^e]:-vec[i^e];
//SUPERSET convolution
for(int e=1;e<n;e*=2)for(int i=0;i<n;++i)
  if(i&e)vec[i^e]+=tp?vec[i  ]:-vec[i  ];
```

5.4.3 divisibility convolution

This allows to do $C(k) = \sum_{k=gcd(i,j)} A_i B_j$ and $C(k) = \sum_{k=lcm(i,j)} A_i B_j$

```
/* Author: Oscar Vargas Pabon
GCD_CONV tested in https://judge.yosupo.jp/problem/gcd_convolution
LCM_CONV tested in https://codeforces.com/gym/105053/problem/G
                                    https://judge.yosupo.jp/problem/
                                        lcm_convolution
Note it wont work well for A*A
Adapted from https://codeforces.com/blog/entry/119082
*/ template<typename T> struct div_conv{
        vector<int> prm; constexpr div_conv(int limit){
                vector<bool> crb(limit,1);
                for(int i=2;i<limit;++i){
                        if(crb[i])prm.push_back(i);
                        for(int p:prm){if(p*1ll*i>=limit)break;
                                crb[i*p]=1;if(i%p==0)break; } }
        } void zeta_mult(vector<T> &vc)const{
                // vc'_x = sum_{x|y} vc_y
                const int n=(vc.size()-1);
                for(int p:prm){ if(p>n)break;
                        for(int i=n/p;i;--i) vc[i]+=vc[i*p]; }
        } void mobi_mult(vector<T> &vc)const{
                // vc_x = sum_{x|y} vc'_y
                const int n=(vc.size()-1);
                for(int p:prm){ if(p>n)break;
                        for(int i=1;i*p<=n;++i)vc[i]-=vc[i*p]; }
        } void gcd_conv(vector<T>&A,vector<T>&B)const{
                // I assume |A|=|B|
                // A'_x = sum_{x=gcd(u,v)} A_u B_v
                zeta_mult(A);zeta_mult(B);
                for(int i=0;i<int(A.size());++i)A[i]*=B[i];
                mobi_mult(A);
        } void zeta_div(vector<T>&vc)const{
                // vc_x = sum_{y|x} vc'_y
                const int n=(vc.size()-1);
                for(int p:prm){ if(p>n)break;
                        for(int i=1;i*p<=n;++i)vc[i*p]+=vc[i]; }
        } void mobi_div(vector<T>&vc)const{
                // vc'_x = sum_{y|x} vc_y
                const int n=(vc.size()-1);
                for(int p:prm){ if(p>n)break;
                        for(int i=n/p;i;--i) vc[i*p]-=vc[i]; }
        } void lcm_conv(vector<T>&A,vector<T>&B)const{
                // I assume |A|=|B|
                // A'_x = sum_{x=lcm(u,v)} A_u B_v
                zeta_div(A);zeta_div(B);
                for(int i=0;i<int(A.size());++i)A[i]*=B[i];
                mobi_div(A); }
}; const div_conv<mint> dc(1e6);
```

5.5 extended gcd

Remember there exist __gcd(a,b); and its usable

```
/*
Plagiado epicamente de cp-algorithms

se me olvido como funciona la extendida de euclides.
-------------------UNFINISHED
g:=gcd(a,b); WLOG a>=b; a=(a/b)+(a%b); then

        g=x*a+y*b
        g=x*((a/b)+(a%b))+y*b
        g=
-------------------
```

```
*/
template<typename tgcd>
tgcd gcd(tgcd a, tgcd b, tgcd& x, tgcd& y) {
        // a*x+b*y==gcd(a,b);
    if (b == tgcd(0)) {
        x = 1;
        y = 0;
        return a;
    }
    tgcd x1, y1;
    tgcd d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}
```

5.6 Xor Basis

```
const int lgi=30; // this impl works entirely by noticing that
// I only have control on 'active' bits. The 'time' is assumed
struct XorB{ vector<pair<int,int>> bs;//to be increasing
        XorB(){bs=vector<pair<int,int>>(lgi,{0,-1});}
        int redu(int x,int y){//reduces element x in times
                for(int i=lgi-1;i>=0;--i)if(bs[i].second>=y)// >=y
                        x=min(x,x^bs[i].first);
                return x;
        } void add(int x,int y){ // add element x in time y
                pair<int,int> act={x,y};
                for(int i=lgi-1;i>=0;--i)if((act.first>>i)&1){
                        if(bs[i].second>act.second)act.first^=bs[i].first;
                        else{ bs[i].first^=act.first;
                                swap(bs[i],act); } }
        } // note that ord(kth(k,y),y)==k and kth(ord(x,y),y)==x
        int kth(int k,int y){
                // returns which is the kth vector (in increasing order) of
                int msk=0;for(int i=0;i<lgi;++i)if(bs[i].second>=y) {
                        msk|=(k&1)<<i; k>>=1;// the span of bs(>=y)
                } int rs=0;for(int i=lgi-1;i>=0;--i)if(bs[i].second>=y){
                        if(((rs>>i)&1)^((msk>>i)&1))rs^=bs[i].first;
                } return rs;
        } int ord(int x,int y){
                // returns which kth does x have on the span of bs(>=y)
                int k=0,e=0;for(int i=0;i<lgi;++i)if(bs[i].second>=y){
                        k|=((x>>i)&1)<<e;
                        ++e;
                } return k;
        } int fred(int y){int rs=0;// degrees of freedom
                for(int i=0;i<lgi;++i)if(bs[i].second>=y)++rs;
                return rs;
        } int mx(int t){ int rs=0; // max element in span
                for(int i=lgi-1;i>=0;--i)if(tm[i]>=t)
                        rs=max(rs,rs^bs[i]);
                return rs; } };
```

5.7 prime stuff

The code has pollard rho that works in expected $O(\sqrt[4]{n})$ and miller-rabin that we can (?) assume it won't fail as a prime tester.

```
/*
Accepted version of the solution to the Factoring Large Integers problem
Use Factorization with Pollards Rho + Primality de Miller-Rabin
Note: The IO part was cut down by edition

Author: Juan Diego Collazos
Date : 20/5/2025
Problem : https://onlinejudge.org/external/114/11476.pdf

Reference:
Pollards Rho Algorithm for Prime Factorization: https://www.geeksforgeeks.org/
    pollards-rho-algorithm-prime-factorization/
Primality Test | Set 3 (Miller-Rabin) : https://www.geeksforgeeks.org/
    primality-test-set-3-miller-rabin/
*/

#include <bits/stdc++.h>

using namespace std;
using ll = long long;

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

ll gcd(ll a, ll b) {
  while (b != 0) {
    ll t = b;
    b = a % b;
    a = t;
  }
  return a;
}

ll mulmod(ll a, ll b, ll mod) {
  ll ans = 0;
  a %= mod;
  while (b > 0) {
    if (b & 1)
      ans = (ans + a) % mod;
    a = (a * 2) % mod;
    b >>= 1;
  }
  return ans;
}

ll powmod(ll base, ll exp, ll mod) {
  ll ans = 1;
  base %= mod;
  while (exp > 0) {
    if (exp & 1)
      ans = mulmod(ans, base, mod);
    base = mulmod(base, base, mod);
    exp >>= 1;
  }
  return ans;
}

bool is_prime(ll n, int k = 5){
  bool ans;
  if (n < 2)
    ans = false;
  else if (n == 2 || n == 3)
    ans = true;
  else if (n % 2  == 0)
    ans = false;
  else{
    ll r = 0, d = n - 1;
    while (d % 2 == 0){
      d >>= 1;
      r++;
    }
    ans = true;
    int i = 0;
    while(i < k && ans){
      ll a = rng() % (n - 3) + 2;
      ll x = powmod(a, d, n);
      if (x != 1 && x != n -1){
        ll j = 0;
        bool composite = true;
        while(j < r - 1 && composite){
          x = powmod(x, 2, n);
          if (x == n - 1)
            composite = false;
```

```
        ++j;
      }
      if (composite)
        ans = false;
    }
    ++i;
  }
 }
 return ans;
}

ll f(ll x, ll n, ll c){
 return (mulmod(x, x, n) + c) % n;
}

ll pollards_rho(ll n) {
 ll ans;
 if (n % 2 == 0)
   ans = 2;
 else if(is_prime(n))
   ans = n;
 else{
   ans = -1;
   while(ans == -1){
     ll c = rng() % n;
     ll x = 2, y = 2, d = 1;
     while(d == 1) {
       x = f(x, n, c);
       y = f(f(y, n, c), n, c);
       d = gcd(abs(x - y), n);
     }
     if (d != n)
       ans = d;
   }
 }
 return ans;
}

void factor(ll n, vector<ll>& factors){
 if (n != 1){
   if (is_prime(n)){
     factors.push_back(n);
   }else{
     ll k = pollards_rho(n);
     factor(k, factors);
     factor(n / k, factors);
   }
 }
}
```

5.8 Floor Sums

```
// Ill eventually try to understand this shit
///// I think it is the key to aladdin from COCI 2009/2010 that I found in the
      CCPL
/// floor sum shenanigans



// taken from https://asfjwd.github.io/2020-04-24-floor-sum-ap/
// its not safe for cases when a<0 or b<0 xdxdxd
long long FloorSumAP(long long a, long long b, long long c, long long n){}
// calculating \sum_{x=0}^{n} \lfloor (ax+b)/c \rfloor
 if(!a) return (b / c) * (n + 1);
 if(a >= c || b >= c) return ( ( n * (n + 1) ) / 2) * (a / c) + (n + 1) * (b
      / c) + FloorSumAP(a % c, b % c, c, n);
 long long m = (a * n + b) / c;
 return m * n - FloorSumAP(c, c - b - 1, a, m - 1);
}
// taken from  atcoder library
namespace internal{
constexpr long long safe_mod(long long x,long long m){ x%=m; if(x<0)x+=m; return
     x; }
unsigned long long floor_sum_unsigned(unsigned long long n,
                                      unsigned long long m,
                                      unsigned long long a,
                                      unsigned long long b) {
    unsigned long long ans = 0;
    while (true) {
        if (a >= m) {
            ans += n * (n - 1) / 2 * (a / m);
            a %= m;
        }
        if (b >= m) {
            ans += n * (b / m);
            b %= m;
        }

        unsigned long long y_max = a * n + b;
        if (y_max < m) break;
        // y_max < m * (n + 1)
        // floor(y_max / m) <= n
        n = (unsigned long long)(y_max / m);
        b = (unsigned long long)(y_max % m);
        std::swap(m, a);
    }
    return ans;
}

}//namespace internal
long long floor_sum(long long n, long long m, long long a, long long b) {
        // Computes \sum_{i=0}^{n-1} \lfloor \frac{a*i+b}{m} \rfloor in time O(lgn)
    assert(0 <= n && n < (1LL << 32));
    assert(1 <= m && m < (1LL << 32));
    unsigned long long ans = 0;
    if (a < 0) {
        unsigned long long a2 = internal::safe_mod(a, m);
        ans -= 1ULL * n * (n - 1) / 2 * ((a2 - a) / m);
        a = a2;
    }
    if (b < 0) {
        unsigned long long b2 = internal::safe_mod(b, m);
        ans -= 1ULL * n * ((b2 - b) / m);
        b = b2;
    }
    return ans + internal::floor_sum_unsigned(n, m, a, b);
}
```

5.9 Max-plus convolution

Notice that this is a specific case of Minkowski Sum of convex polygons.

```
/*
Author:Oscar Vargas Pabon
Computing: Given A[0..n) and B[0..m) compute
        B[0..n+m-1) where B[k] = max_{i+j=k}{A[i]+B[j]}
The concave version is basically minkowski sum of convex polygons
        but specialized for this specific problem.
Concave version tested in: https://atcoder.jp/contests/ndpc/tasks/ndpc2026_o

Ill eventually keep looking
*/
template<typename tint>
void assert_concavity(const vector<tint>&A){
        if(A.size()<=1)return; // O(n)
        tint prv=A[1]-A[0];rep(i,2,A.size()){
                const tint delta=A[i]-A[i-1];
                if(delta>prv)assert(string("Esto no es")==string("concavo"));
                if(delta<prv)assert(string("Esto no es")==string("convexo"));
                prv=delta;
```

```
        } }
template<typename tint>
vector<tint> maxplus_convolution_concave(const vector<tint>&A,const vector<
    tint>&B){
        const int n=A.size(),m=B.size();//O(n+m)
        vector<tint> res(n+m-1);
        int aind=0,bind=0;while(aind<n&&bind<m){
                res[aind+bind]=A[aind]+B[bind];
                if(aind+1>=n)++bind;
                else if(bind+1>=m)++aind;
                else if(A[aind+1]-A[aind]>=B[bind+1]-B[bind])++aind;
                else ++bind;
        } return res; }
template<typename tint,tint inf=1e9>
vector<tint> max_plus_convolution_naive(){//O(n*m)
        vector<tint> brute(n+m-1,-inf);rep(i,0,n)rep(j,0,m)
                brute[i+j]=max<tint>(brute[i+j],A[i]+B[j]);
        return brute; }
```

5.10 sieve

```
/* Author: Oscar Vargas Pabon

Based loosely on https://codeforces.com/blog/entry/54090
Can be extended to calculate other multiplicative functions such as
        phi(p^e)=p^(e-1)(p-1) or ad-hoc stuff (remember I can mantain the value of e
        to calculate certain weird multiplicative functions)
*/
int mu[max_n];void sieve(int n){
        vector<bool>crb(n,0);vector<int>prm;
    crb[1]=mu[1]=1;rep(i,2,n){
        if(!crb[i])prm.pb(i),mu[i]=-1;
        for(int p:prm){if(i*1ll*p>=n)break;
            crb[i*p]=1;
            if(i%p==0){mu[i*p]=0;break;}
            mu[i*p]=mu[i]*mu[p];
        }
    }
}
```

6 Combinatorics

Muchas de estas propiedades fueron sacadas de https://cp-algorithms.com/combinatorics

6.1 Properties

Remember $n! = n \cdot (n-1)!$ and $(n)_k = \frac{n!}{(n-k)!} = n(n-1)\cdots(n-k+2)(n-k+1)$ (this last one generalizes for $n \notin \mathbb{N}$).

6.1.1 n Choose k

Remember that it can be calculated by the recursive formula, by precomputing factorials modulo p, or in $min(k, n-k)$ by calculating $(k!)^{-1} \cdot (n)_k (mod p)$.

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

$$\sum_{i=0}^{n} x^i \binom{n}{i} = (x+1)^n$$

$$\sum_{m=0}^{n} \binom{m}{k} = \binom{n+1}{k+1}$$

$$\sum_{k=0}^{m} \binom{n+k}{k} = \binom{n+m+1}{m}$$

$$\sum_{k=0}^{n} \binom{n}{k}^2 = \binom{2n}{n}$$

$$\sum_{i=0}^{n} i \binom{n}{i} = n \cdot 2^{n-1}$$

Wilson's theorem:

$$(p-1)! \equiv -1 (\text{mod p})$$

Lucas theorem:

$$\binom{n}{k} \equiv \prod \binom{n_i}{k_i} (\text{mod p})$$

where $n = \sum n_i \cdot p^i$ and $k = \sum k_i \cdot p^i$

Vandermonde's identity:

$$\sum_{k=0}^{n} \binom{m}{k}\binom{n}{r-k} = \binom{m+n}{r}$$

6.1.2 Stirling God

Stirling number of second kind $\left\{ {n \atop k} \right\}$ counts the amount of ways to partition n unlabelled objects in k non-empty subsets.

$$\left\{ {n+1 \atop k} \right\} = k \left\{ {n \atop k} \right\} + \left\{ {n \atop k-1} \right\}$$

$$\left\{ {n \atop k} \right\} = \frac{1}{k!} \sum_{i \geq 0} (-1)^{k-i} \binom{k}{i} i^n$$

$$\left\{ {n \atop k} \right\} \equiv \binom{z}{w} (mod 2)$$

Where $z = n - \lceil \frac{k+1}{2} \rceil$ and $w = \lfloor \frac{k-1}{2} \rfloor$.

As bell numbers B_n count the amount of ways to partition n elements then

$$B_n = \sum_{i=0}^{n} \left\{ {n \atop k} \right\}$$

Stirling number of the first kind (eventually).

6.1.3 Formal Power Series

Note: This is still poor in properties.

$$(1+F)^\lambda = \sum_{n \geq 0} \binom{\lambda}{n} F^n$$

Remember it generalizes to $\lambda \notin \mathbb{N}$ by using the $(n)_k$ definition of $\binom{n}{k}$.

$$(1-x)^{-1} = \sum_{n \geq 0} x^n \text{ for } 0 \leq x < 1$$

$$e^x = \sum_{n\geq 0} \frac{x^n}{n!}$$

$$(1-x)^{-k} = \sum_{n\geq 0} \binom{n+k-1}{n} x^n$$

$$-ln(1-x) = \sum_{x\geq 1} \frac{x^n}{n}$$

$$F \cdot G = \sum_{n\geq 0} \left(\sum_{i=0}^{n} f_i g_{n-i} \right) x^n \qquad \text{for OGF}$$

$$= \sum_{n\geq 0} \left(\sum_{i=0}^{n} \binom{n}{i} f_i g_{n-i} \right) x^n \qquad \text{for EGF}$$

$$F^k = \sum_{n\geq 0} \left(\sum_{i_1+i_2+\cdots+i_k=n} \prod_{j=1}^{k} f_{i_j} \right) x^n \qquad \text{for OGF}$$

$$= \sum_{n\geq 0} \left(\sum_{i_1+i_2+\cdots+i_k=n} \binom{n}{i_1 \cdot i_2 \cdots i_k} \prod_{j=1}^{k} f_{i_j} \right) x^n \qquad \text{for EGF}$$

$$\sum_{n\geq 0} \sum_{k\geq 0} \binom{n}{k} x^n y^k = \frac{1}{1-x-xy}$$

6.1.4 Modular Arithmetic

$\phi(n)$ is the amount of coprime elements to n in the range $[0, n)$.

$$\phi(a \cdot b) = \phi(a)\phi(b)$$

$$\phi(p) = p - 1$$

$$\phi(m) = m \prod_{i=1}^{k} (1 - \frac{1}{p_i}) \text{ where } m = \prod_{i=0}^{k} p_i^{e_i}$$

$$a^{\phi(m)} \equiv 1 (\text{mod } m)$$

6.1.5 Chinese Remainder Theorem

The problem is to find x such that

$$\begin{cases} x \equiv a_1 (\text{mod } m_1) \\ \vdots \\ x \equiv a_k (\text{mod } m_k) \end{cases}$$

and all m_i's are pairwise coprime.

$$R^* = \sum_{i=1}^{k} a_i \left[inv_{m_i} \left(\prod_{j\neq i} m_j \right) \prod_{j\neq i} m_j \right]$$

Remember R^* works modulo $\prod m_j$.

6.1.6 Lagrange Interpolation

Find lowest degree polinome that satisfies $P(x_i) = y_i \forall_i$ given $[(x_1, y_1), \cdots, (x_k, y_k)]$.

$$P = \sum_{i=0}^{k} \left(\frac{\prod_{j\neq i}(x_j - x)}{\prod_{j\neq i}(x_j - x_i)} \right) y_i$$

Remember that if I only want P to evaluate on a specific x, then I can just make x in the equation as the value to avoid building the explicit polinomial.

6.1.7 Catalan numbers

Catalan Numbers

$$C_0 = C_1 = 1$$

$$C_i = \sum_{k=0}^{i-1} C_k C_{i-1-k}$$

$$C_i = \frac{1}{n+1} \binom{2n}{n}$$

$$C_i = \frac{4i-2}{i+1} C_{i-1}, C_0 = 1$$

6.2 Techniques

Principle of Inclusion-Exclusion (PIE)

Remember there are more and more ways to write PIE that can help with other stuff (I think one of those is mobius function, but more on that when im good).

$$|\cup S_i| = \sum_{\emptyset \neq J \in \{1..N\}} (-1)^{|J|-1} |\cap_{i\in J} S_i|$$

Stars and bars

$$\binom{n+k-1}{n}$$

Number of ways to add n identical objects into k labeled boxes.

Armonica: $\sum_{k=1}^{n} \frac{1}{k} = O(lgn)$

7 Geometry

7.1 geomTemplate

```
/*
Autor: Oscar Vargas Pabon

GRAFOS PLANARES:::::
c:componentes conexos ; R:regiones
m:aristas; n:nodos
R+n=m+c+1

Probado en UVA 13117 y 11894
*/
const double eps=1e-8;
template<typename Tpt> struct Point{
        Tpt x,y;
        Point()=default;
        Point(Tpt x, Tpt y ) : x(x),y(y){};
        Point rot90(){return{-y,x};}

        // NOTA: puedo querer prescindir del sqrt
        Tpt norm()const{return sqrt(x*x+y*y);}
        Tpt norm2()const{return x*x+y*y;}
```

```
        Point scale(Tpt v)const{return{v*x,v*y};}
        Point operator -()const{return{-x,-y};}
        Point operator -(const Point&o)const{return{x-o.x,y-o.y};}
        // suma de vectores
        Point operator +(const Point&o)const{return{x+o.x,y+o.y};}

        // point product; 0->ortogonal; +->same dir ; - ->opposite dir
        Tpt operator *(const Point&o)const{return x*o.x+y*o.y;}
        // cross product; 0->colinear; +->left side; - ->right side
        Tpt operator %(const Point&o)const{return x*o.y-y*o.x;}

// abs(v%u)==u.norm2()*v.norm2()*sin(theta)
// (v*u)==u.norm2()*v.norm2()*cos(theta)
// v%u == area paralelogramo delimitado por u y v
        bool operator<(const Point &o)const{return make_pair(x,y)<make_pair(o.
            x,o.y);}
        bool operator==(const Point&o)const{return x==o.x&&y==o.y;}
        bool operator!=(const Point&o)const{return!(*this==o);}
        friend ostream & operator << (ostream &out, const Point<Tpt> &p){ out
            << "("<<p.x<<","<<p.y<<")"; return out; }
}; typedef Point<double> Pt;

template<typename Tpt> struct Line{
        Point<Tpt> p,v;// L(t)=p+tv
        Line()=default;

        // L(0)=a; L(1)=b; L(t) 0<=t<=1 esta en el segmento ab
        Line(Point<Tpt> a,Point<Tpt> b):p(a),v(b-a){};
        Point<Tpt> eval(Tpt t) { return p+v.scale(t);}
        // hallo t tal que L(t)== interseccion entre this y o
        // recordar que puedo trabajar en enteros hasta la division
        Tpt inter(const Line&o){return{((p-o.p)%o.v)/(v%o.v)};}

        // dados v,u vectores; el valor h donde {0,v(h),u} forman un triangulo
             rectangulo
        // con con angulo recto A{o,v(h),u} =90 es tal que u*v=h*(v*v)
        friend ostream & operator << (ostream &out, const Line<Tpt> &l){ out
            << "<L(t)="<<l.p<<"+t"<<l.v<<">"; return out; }
};typedef Line<double> Ln;
```

7.2 Convex Hull (2D)

```
/*
Autor: Oscar Vargas Pabon

Works in O(n lgn)
Think on impl with vector or smthng
There are stuff / versions that are incremental

Tested in https://codeforces.com/problemset/problem/1017/E
          https://open.kattis.com/problems/convexhull

I assume from my template:
#define rep(i,strt,end) for(int i = strt ; i !=int(end) ; (int(strt)<int(end))
    ?++i:--i )
#define pb push_back
#define pob pop_back
*/

vector<Pt> convex_hull(Pt *arr,int n){
        sort(arr,arr+n);
        vector<Pt> up,dwn;
        auto crss=[](const Pt&a,const Pt &b,const Pt &bs){return (a-bs)%(b-bs)
            ;};
        rep(i,0,n){
                Pt &ac=arr[i]; int pz=up.size(),dz=dwn.size();
                while(pz>1&& crss(ac,up[pz-2],up[pz-1])>=0 ){
                        --pz; up.pob();
                }
                if(pz>=1&&up.back().x==ac.x&&up.back().y<ac.y)up.pob();
                if(up.empty()||up.back().x<ac.x||up.back().y<ac.y)up.pb(ac);

                while(dz>1&& crss(ac,dwn[dz-2],dwn[dz-1])<=0 ){
                        --dz; dwn.pob();
                }
                if(dz>=1&&dwn.back().x==ac.x&&dwn.back().y>ac.y)dwn.pob();
                if(dwn.empty()||dwn.back().x<ac.x||dwn.back().y>ac.y)dwn.pb(ac
                    );
        }
        vector<Pt> res=dwn;
        rep(i,up.size()-1,-1){
                if( !(res.back()==up[i]) ) res.pb(up[i]);
        }
        // idebug(up);idebug(dwn);
        if(int(res.size())>1&&res.front()==res.back())res.pob();
        return res;
}


vector<int> hullInd(const vector<Pt>& v) {
        //adapted from https://github.com/bqi343/cp-notebook

        int ind = int(min_element(all(v))-begin(v));
        vector<int> cand, C={ind};
        rep(i,0,v.size()) if (v[i] != v[ind]) cand.pb(i);
        sort(all(cand),[&](int a, int b) {
                // sort by angle, tiebreak by distance
                Pt x = v[a]-v[ind], y = v[b]-v[ind]; Tpt t = x%y;
                return t != 0 ? t > 0 : x*x < y*y;
        });
        auto crss=[](const Pt&a,const Pt &b,const Pt &bs){return (a-bs)%(b-bs)
            ;};
        for (int c:cand){
                while (int(C.size()) > 1 && crss(v[C.back()],v[c],v[C[C.size()
                    -2]]) <= 0)
                        C.pob();
                C.pb(c); }
        return C;
}
vector<Pt> hull(const vector<Pt>& v) {
        vector<int> w = hullInd(v); vector<Pt> res;
        for(int t:w) res.pb(v[t]);
        return res; }
```

8 linAlg

8.1 LP Simplex

Recordar que la implementacion de simplex requiere que el sistema este en 'slack' form

$$\begin{cases} c_{1,1}x_1 + c_{1,2}x_2 + \cdots + c_{1,k}x_k = y_1 \\ \vdots \\ c_{m,1}x_1 + c_{m,2}x_2 + \cdots + c_{m,k}x_k = y_m \end{cases}$$

$$\forall_i x_i \geq 0$$

Condiciones del estilo $\sum_i c_i \cdot x_i \leq y$ se convierten a 'slack' form anadiendo una 'slack' variable s que tiene coeficiente 0 en todas las filas (y objetivo) y 1 en la fila actual. El sistema queda como $\sum_i c_i \cdot x_i + s = y$ para $s \geq 0$. Las condiciones del estilo $\sum_i c_i \cdot x_i \geq y$ son convertidas a $\sum_i (-c_i) \cdot x_i \leq -y$ (y luego a 'slack' form como lo anterior).

Lo use en el gym 101242 I (Road Times ICPC WF 2016).

```
/*
Tomado del notebook de DebureoMinkyuParty
https://github.com/koosaga/DeobureoMinkyuParty
*/
using T = long double;
const int N = 410, M = 30010;
const T eps = 1e-7;
```

```
int n, m;
int Left[M], Down[N];
// time complexity: exponential. fast O(MN^2) in experiment. dependent on the
    modeling.
// Ax <= b, max c^T x.
T a[M][N], b[M], c[N], v, sol[N];
bool eq(T a, T b) { return fabs(a - b) < eps; }
bool ls(T a, T b) { return a < b && !eq(a, b); }
void init(int p, int q) {
        n = p; m = q; v = 0;
        for(int i = 1; i <= m; i++){
                for(int j = 1; j <= n; j++) a[i][j]=0;
        }
        for(int i = 1; i <= m; i++) b[i]=0;
        for(int i = 1; i <= n; i++) c[i]=sol[i]=0;
}
void pivot(int x,int y) {
        swap(Left[x], Down[y]);
        T k = a[x][y]; a[x][y] = 1;
        vector<int> nz;
        for(int i = 1; i <= n; i++){
                a[x][i] /= k;
                if(!eq(a[x][i], 0)) nz.push_back(i);
        }
        b[x] /= k;
        for(int i = 1; i <= m; i++){
                if(i == x || eq(a[i][y], 0)) continue;
                k = a[i][y]; a[i][y] = 0;
                b[i] -= k*b[x];
                for(int j : nz) a[i][j] -= k*a[x][j];
        }
        if(eq(c[y], 0)) return;
        k = c[y]; c[y] = 0;
        v += k*b[x];
        for(int i : nz) c[i] -= k*a[x][i];
}
// 0: found solution, 1: no feasible solution, 2: unbounded
int lp_solve() {
        for(int i = 1; i <= n; i++) Down[i] = i;
        for(int i = 1; i <= m; i++) Left[i] = n+i;
        while(1) { // Eliminating negative b[i]
                int x = 0, y = 0;
                for(int i = 1; i <= m; i++) if (ls(b[i], 0) && (x == 0 || b[i]
                    < b[x])) x = i;
                if(x == 0) break;
                for(int i = 1; i <= n; i++) if (ls(a[x][i], 0) && (y == 0 || a
                    [x][i] < a[x][y])) y = i;
                if(y == 0) return 1;
                pivot(x, y);
        }
        while(1) {
                int x = 0, y = 0;
                for(int i = 1; i <= n; i++)
                if (ls(0, c[i]) && (!y || c[i] > c[y])) y = i;
                if(y == 0) break;
                for(int i = 1; i <= m; i++)
                if (ls(0, a[i][y]) && (!x || b[i]/a[i][y] < b[x]/a[x][y])) x =
                    i;
                if(x == 0) return 2;
                pivot(x, y);
        }
        for(int i = 1; i <= m; i++) if(Left[i] <= n) sol[Left[i]] = b[i];
        return 0;
}
```

8.2 matrix primitives

```
/* Author: Oscar Vargas Pabon
Partially tested in https://qoj.ac/contest/1814/problem/843
It may be slightly slow
*/
typedef vector<vector<mint>> mat;
template<typename tint>mat transpose(const mat&m){
    const int n=m.size(),k=m[0].size();
    vector<vector<tint>>tran(k,vector<tint>(n));
    for(int i=0;i<n;++i)for(int j=0;j<k;++j)tran[j][i]=m[i][j];
    return tran;
} template<typename tint>mat mat_mult(const mat&m1,const mat&m2){
    const int n=m1.size(),m=m2.size(),k=m2[0].size();
    assert(int(m1[0].size())==m);
    vector<vector<mint>> res(n,vector<mint>(k,tint(0)));
    for(int i=0;i<n;++i)for(int j=0;j<m;++j)for(int h=0;h<k;++h)res[i][h]+=m1[
        i][j]*m2[j][h];
    return res;
} template<typename tint>tint trace(const mat&m){
    tint res=0;for(int i=0;i<int(m.size());++i)res+=m[i][i];
    return res;
}template<typename tint>tint determinant(mat m){
    const int n=m.size();assert(int(m[0].size())==n);
    tint det=1;for(int i=0;i<n;++i){
        for(int j=i;j<n;++j)if(m[i][j]!=tint(0)){
            swap(m[i],m[j]); if(j!=i)det=-det;
            break;
        }det*=m[i][i];if(det==tint(0))return tint(0);
        {tint ac=m[i][i].inv();for(int j=i;j<n;++j)m[i][j]*=ac;}
        for(int j=i+1;j<n;++j)if(m[j][i]!=tint(0)){
            tint ac=-m[j][i];for(int k=i;k<n;++k)m[j][k]+=m[i][k]*ac;
        }
    }return det;
}template<typename tint>
tint kth_lin_recurrence(uint64_t e,const vector<tint>&recur,const vector<tint
    >&init){
// I assume the recurrence coefficients to come as
// a(n)=\sum_{i=0}^{n-1} recur(n-1-i)a(n-1-i) for the sequence a
    const int n=recur.size();assert(int(init.size())==n);
    if(e<n)return init[e];
    vector<vector<tint>> A(n,vector<tint>(n,tint(0))),B=A;
    for(int i=0;i<n;++i)B[i][i]=1;
    for(int i=1;i<n;++i)A[i-1][i]=1;
    A.back()=recur; e-=n; while(e){
        if(e&1ull)B=mat_mult<tint>(B,A);
        e>>=1;A=mat_mult<tint>(A,A);
    } return mat_mult<tint>(B,transpose<mint>(vector<vector<tint>>(1,init))).
        back().back();
}
```

8.3 slae

When dealing with modulo 2 stuff bitset can also carry hard

```
// https://cp-algorithms.com/linear_algebra/linear-system-gauss.html
// The input to the function gauss is the system matrix
// a. The last column of this matrix is vector
// b
const double EPS = 1e-9;
const int INF = 2; // it doesn't actually have to be infinity or a big number
int gauss (vector < vector<double> > a, vector<double> & ans) {
    int n = (int) a.size(), m = (int) a[0].size() - 1;
    vector<int> where (m, -1);
    for (int col=0, row=0; col<m && row<n; ++col) {
        int sel = row; for (int i=row; i<n; ++i)
            if (abs (a[i][col]) > abs (a[sel][col]))
                sel = i;
        if (abs (a[sel][col]) < EPS) continue;
        for (int i=col; i<=m; ++i) swap (a[sel][i], a[row][i]);
        where[col] = row; for (int i=0; i<n; ++i)
            if (i != row) {
                double c = a[i][col] / a[row][col];
                for (int j=col; j<=m; ++j)
                    a[i][j] -= a[row][j] * c;
            }
```

```
        ++row;
    } ans.assign (m, 0);
    for (int i=0; i<m; ++i) if (where[i] != -1)
            ans[i] = a[where[i]][m] / a[where[i]][i];
    for (int i=0; i<n; ++i) {
        double sum = 0; for (int j=0; j<m; ++j)
            sum += ans[j] * a[i][j];
        if (abs (sum - a[i][m]) > EPS) return 0;
    } for (int i=0; i<m; ++i) if (where[i] == -1)
            return INF;
    return 1; }
```

9 Cosas random

9.1 Calcular $\lfloor log_2 x \rfloor$

Nota, tambien está en Fenwick Tree.

En C++

```
sizeof(int)*8 - __builtin_clz(x)-1;
```

En Python

```
x.bit_length()-1
```

9.2 My tipsNtricks stuff

Remember the trick of the transpose graph in $O((n+m)lgn)$ using the set to keep track of the unvisited ones.

```
/* Autor: Oscar Vargas Pabon
These are various tricks to be used at will
*/ // this pragmas optimize somehow bitset
#pragma GCC optimize("O3", "unroll-loops")
#pragma GCC target("avx2", "popcnt")

// iterating over ranges [l_i,r_i]\subseteq[L,R] such that (assuming integer
    division)
//////   \forall_{j,h\in[l_i,r_i]} n/j = n/h
/// can be seen in https://codeforces.com/problemset/problem/2072/G
void diviter(int n, int L, int R ){
        int l = L,r; while( l <= R ){
                int nh = n/l;
                r = min(R,n/nh);

                /// do stuff with the range i \in [l,r] n/i = nh

                l=r+1;
        }
} // ham-path
// phi(nd,msk) = strt \in N(nd) OR OR_{e\in N(nd)} phi(nd,msk|(1<<nd))
// In cases where I'm only looking for cycles containing msk, then I will
    start
// from every node; however, I don't need to erase the memory. The reason is
    that
// if such cycle exists I can find it with any strt \in msk, if there is none it
    won't
// matter which strt \in msk I choose.
// This idea can be seen in https://codeforces.com/problemset/problem/1804/E

/////// XOR weird properties
// https://codeforces.com/blog/entry/141285
// a+b= a^b + 2(a&b)
// a^b <= 2*max(a,b)
// a^b=(a|b)-(a&b)
// a-b <= a^b <= a+b
// A[0..n) such that A[i]<=A[i+1], then min_{i!=j}(A[i]^A[j])=min_i(A[i]^A[i
    +1])

/////// Range Xor Shenanigans (assuming range is [l,r])
// Question: Is subset in range interesting? Is it doable in O(1)?
// I used this in industrial nim and in https://codeforces.com/problemset/
    problem/2056/F2

// calculating xor in range
int xor_range(int m){
        //calculates XOR_{x=0}^{m} x in O(1). Observe it includes m.
        int res=(m&1)?0:m; return res^((m+1)>>1)&1;
} // calculating xor in range V.2
template<typename tXor> tXor xor_range_V2(tXor n){
        vector<tXor> _p = {n, 1, n + 1, 0}; return _p[n % 4];
}// This one follows by induction or showing \oplus_{i=0}^{4n-1} i = 0
// Close inspection shows both versions do the same
// https://codeforces.com/blog/entry/141285

// The idea is that if I fix the bits that are always set, then I can exclude
    them
// x= 0010100
// y= ??1?1??
// meaning that if I exclude the 1's it becomes a xor_range problem
// this involves calculating the new m (since the actual m may not me superset
    )
//    and re-adding again the 'erased' bits after xor_range
int super_set(int m,int x){
        //calculates XOR_{y=0}^{m}(y&x==x)*y in O(lgU)
        if(x>m)return 0;
        //Finding last superset in range
        int lgi=ilog2(m-1),lst=x;
        rep(e,lgi,-1)if((lst|(1<<e))<m)lst|=1<<e;
        int neo=0,xx=x,e=0;
        while(lst){//Reducing to xor_range
                if(!(xx&1))neo|=(lst&1)<<e++;
                lst>>=1;xx>>=1;
        } int skw=xor_range(neo);xx=x;
        int res=0; e=0;
        while(skw){//reconstruct answer from xor_range
                if(!(xx&1)){ res|=(skw&1)<<e;
                        skw>>=1;
                } xx>>=1; ++e;
        } // add the bits in x (which are always set)
        if(x)res|=((neo+1)&1)*x;
        return res; }
```

9.3 template

9.3.1 c++

```
/*
 --------
|    ___ |
|   ,',.(`|
| :   ``  |
| :)  _   (|
|   `:-)_,|
|--------|

Autor: Oscar Vargas Pabon
Fecha:

*/
//#pragma GCC optimize("O3")
//#define NDEBUG
#include <bits/stdc++.h>
#include <cassert>

typedef long long lint;
// __uint128_t, __int128_t, uint64_t, int64_t, uint32_t,int32_t
using namespace std;
#ifdef OSVARP
    #include<sys/resource.h>
#else
    #define cerr for(;false;) cerr
```

```
#endif
template <typename t1,typename t2> std::ostream &operator<<(ostream &os, const
    std::pair<t1,t2>&pr){return os<<"("<<pr.first<<";"<<pr.second<<")";};
template <typename T> std::ostream &operator<<(ostream &os, const std::vector<
    T>&vc){for(auto ac:vc)os<<ac<<'-';return os;};
template <typename T> std::ostream &operator<<(ostream &os, const std::set<
    T>&vc){for(auto ac:vc)os<<ac<<'-';return os;};
template <typename t1,typename t2> std::ostream &operator<<(ostream &os, const
    std::map<t1,t2>&vc){for(auto ac:vc)os<<ac<<'-';return os;};
#define debug(args...) { string _s = #args; replace(_s.begin(), _s.end(), ',',
    '-'); stringstream _ss(_s); istream_iterator<string> _it(_ss); raw_debug(
    _it, args);}
void raw_debug(istream_iterator<string> it) {cerr<<endl;assert(it==it);}
template<typename T, typename... Args>
void raw_debug(istream_iterator<string> it, T a, Args... args) { cerr <<"<"<<
    *it << "->" << a << ">-"; raw_debug(++it, args...); }
#define adebug(ar,n) {cerr<<'['<<#ar<<']';for(int my_imp_ind=0;my_imp_ind<n;++
    my_imp_ind)cerr << '-' << ar[my_imp_ind]; cerr << endl;}

#define rep(i,strt,end) for(int i = strt ; i !=int(end) ; (int(strt)<int(end))
    ?++i:--i )
#define rall(vec) vec.rbegin(), vec.rend()
#define all(vec) vec.begin(), vec.end()
#define eb emplace_back
#define pb push_back
#define pob pop_back
#define pf push_front
#define pof pop_front

std::mt19937_64 rng_64( std::chrono::steady_clock::now().time_since_epoch().
    count() );
constexpr int ilog2( int num ) { return 8*sizeof(int) - __builtin_clz( num ) -
    1; }
template<typename tpow,typename texp=int64_t> constexpr tpow mpow(tpow x,
    uint64_t e,tpow m){tpow res=1;while(e){if(e&1)res=(texp(res)*x)%m;e>>=1;x
    =(texp(x)*x)%m;}return res;}

const int max_n = 1e6;
int a[max_n], b[max_n];

void solve() {

} int32_t main(){
        ios_base::sync_with_stdio(false);
    cin.tie(NULL);
        cout << setprecision(12) << fixed;
#ifdef OSVARP
    auto start = chrono::high_resolution_clock::now();
#endif

    int t = 1;
    cin >> t;
    ++t; while ( --t ) solve();

#ifdef OSVARP
    auto end = chrono::high_resolution_clock::now();
    struct rusage usage; getrusage(RUSAGE_SELF, &usage);
    cerr << "\n<Execution time: "
        << chrono::duration_cast<chrono::milliseconds>(end - start).count()
        << " ms>\n<Memory used: "
        << usage.ru_maxrss << " kilobytes>" << endl;
#endif
    return 0; }
```

9.3.2 Python

Python es una calculadora glorificada. Pero tiene algunos usos...

```
## necesities
import sys
sys.set_int_max_str_digits(10**9)
sys.setrecursionlimit(10**9)
## Examples
linea = sys.stdin.readline().strip()
arr = list(map(int, sys.stdin.readline().split()))
sys.stdout.write("Esto es una salida rapida\n")
## leer hasta EOF
for linea in sys.stdin:
    datos = linea.strip().split()
    if not datos: continue
    nums = list(map(int, datos))
```

9.4 Exhaustive testing

```
# https://codeforces.com/blog/entry/102287
# linux
for i in {1..100}; do
    echo "on test $i" ./gen.out > input.txt ./a.out < input.txt > output.txt if [ $? -ne 0 ]; then
        echo "Something wrong"
    cat input.txt
    cat output.txt
    exit 1
    fi
    #./naive.out < input.txt > answer.txt
    #diff output.txt answer.txt || break
done
echo "All ok"
```

```
REM https://codeforces.com/blog/entry/102287
REM WINDOWS
@echo off
for /l %%i in (1, 1, 100) do (
    echo %%i

    gen.exe > input.txt
    a.exe < input.txt > output.txt
    stl.exe < input.txt > answer.txt
    fc output.txt answer.txt || goto :out
) :out
echo on
```